Hochschule für Technik
und Wirtschaft Berlin
University of Applied Sciences

Optimierungsstrategien für CLIP-basierte Text-zu-Video-Retrieval-Systeme

Bachelorarbeit

Name des Studiengangs

Internationaler Studiengang Medieninformatik

Fachbereich 4
Informatik, Kommunikation und Wirtschaft

vorgelegt von

Ole Marwede

Datum:

06.08.2026

Erstgutachter_in: Prof. Klaus Jung

Zweitgutachter_in: Bruno Schilling

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Motivation und Hintergrund . . . 1
1.2 Problemstellung . . . 1
1.3 Zielsetzung und Aufgabenstellung . . . 2
1.4 Aufbau der Arbeit . . . 2

2 Grundlagen **4**

2.1 Information und Video Retrieval . . . 4
2.1.1 Grundlagen Information Retrieval . . . 4
2.1.2 Grundlagen Video Retrieval . . . 4
2.2 Neuronale Netze und Vision Language Modelle . . . 5
2.2.1 Künstliche Neuronale Netzwerke . . . 5
2.2.2 Visuelle Feature-Extraktion . . . 7
2.2.3 Vision Language Modelle . . . 8
2.2.4 (Open-)CLIP . . . 9
2.3 Verwendete Algorithmen und Verfahren . . . 10
2.3.1 Frame-Sampling und Keyframe Extraktion . . . 10
2.3.2 Von Frame- zu Video-Ähnlichkeit . . . 11
2.3.3 Modell-Interpolation . . . 11
2.3.4 Effiziente Vektorsuche . . . 12
2.4 Verwandte Arbeiten . . . 13
2.4.1 VBS . . . 13
2.4.2 WiSE-FT . . . 13
2.4.3 Open-VCLIP . . . 14
2.4.4 CLIP-Optimization for Video Search . . . 14

3 Konzeption **15**

3.1 Anforderungen und Zielstellung . . . 15
3.1.1 Zielstellung und Forschungsfragen . . . 15
3.1.2 Funktionale Anforderungen . . . 16
3.1.3 Nicht-funktionale Anforderungen . . . 17
3.2 Evaluationsdesign . . . 17
3.2.1 Datensatzwahl . . . 17
3.2.2 Metriken . . . 18
3.2.3 Evaluationsablauf . . . 19

3.3 Auswahl der Ansätze . . . 20
3.3.1 Frame-Sampling-Strategien . . . 20
3.3.2 Modellauswahl . . . 21
3.3.3 Fine-Tuning . . . 21
3.3.4 Gewichtsinterpolation . . . 22
3.3.5 Vektorsuche . . . 23
3.4 Systementwurf . . . 23
3.4.1 Technologien . . . 23
3.4.2 Architektur . . . 24

4 Implementierung **27**
4.1 Übersicht . . . 27
4.2 Vorverarbeitung . . . 28
4.2.1 Feste Abtastraten . . . 28
4.2.2 TransnetV2 . . . 28
4.2.3 Datenvolumen . . . 29
4.2.4 Charades . . . 29
4.3 Modellerzeugung . . . 30
4.3.1 Aufbereitung der Trainingsdaten . . . 30
4.3.2 Durchführung des Fine-Tunings . . . 30
4.3.3 Auswahl der Epochen . . . 31
4.3.4 Gewichtsinterpolation . . . 33
4.4 Indexierung und Suche . . . 33
4.4.1 Bild-Embeddings und Index . . . 33
4.4.2 Anfrageverarbeitung . . . 34
4.5 Evaluation und Automatisierung . . . 34
4.5.1 Evaluationsablauf . . . 34
4.5.2 Automatisierung . . . 35
4.6 Benutzeroberfläche . . . 35
4.7 Anpassungen während der Implementierung . . . 36

5 Ergebnisse und Auswertung **38**
5.1 Überblick und Darstellungsweise . . . 38
5.2 Fine-Tuning . . . 38
5.3 Interpolationsfaktor . . . 39
5.4 Frame-Sampling . . . 44
5.5 Einfluss der Basismodelle . . . 46
5.6 Verhalten unter Distribution-Shift . . . 47
5.7 Zusammenfassende Bewertung . . . 48

6 Zusammenfassung und Ausblick **50**
6.1 Zusammenfassung . . . 50
6.2 Ausblick . . . 52

A Messergebnisse **55**

Abbildungsverzeichnis **61**

Tabellenverzeichnis **62**

Quelltextverzeichnis **64**

Literaturverzeichnis **65**

Kapitel 1: Einleitung

1.1 Motivation und Hintergrund

Die Menge an täglich produziertem Videomaterial wächst seit Jahren ununterbrochen, sei es in sozialen Medien, in Film und Fernsehen, in der Werbung oder in der Wissenschaft. Mit dieser Menge steigt die Schwierigkeit eine bestimmte Stelle eines Videos in einer großen Sammlung ausfindig zu machen. Die Aufgabe, diese konkrete Stelle zu finden, wird in der Wissenschaft als Known-Item-Search (KIS) bezeichnet und ist eine der zentralen Aufgaben des Video Browser Showdowns (VBS) [25], einem internationalen Wettbewerb für interaktive Videosuche.

Klassische Verfahren, die Inhalte anhand von Metadaten ausfindig machen, stoßen hier an ihre Grenzen, da Metadaten oft fehlen oder unzureichend sind und sich nicht jedes Video manuell annotieren lässt. Einen Durchbruch brachten Vision-Language-Modelle wie CLIP [16], die Bilder und Texte in einen gemeinsamen Embedding-Raum projizieren. Dadurch wird ermöglicht, dass Bilder über Eingabe von natürlicher Sprache gefunden werden können, ohne diese zuvor zu annotieren.

Ein vortrainiertes CLIP-Modell operiert auf einzelnen Bildern und ist nicht auf die zeitliche Struktur von Videos zugeschnitten. Dies wirft die Frage auf, wie ein CLIP-Modell bestmöglich an die interaktive Videosuche angepasst und optimiert werden kann, um Genauigkeit und Effizienz eines Video-Retrieval-Systems zu verbessern.

1.2 Problemstellung

Ein vortrainiertes CLIP-Modell (Zero-Shot) ist robust, aber nicht auf die Eigenschaften von Videodaten zugeschnitten. Durch gezieltes Fine-Tuning auf Videodaten kann die Genauigkeit des Modells verbessert werden, jedoch besteht hier die Gefahr von Overfitting (Überanpassung) und der damit verbundene Verlust an Robustheit. Die lineare Interpolation der Gewichte (WiSE-FT) [27] zwischen dem Zero-Shot-Modell und dem Fine-Tuned Modell verspricht eine Verbesserung der Genauigkeit, ohne die Robustheit zu verlieren. In der Literatur wird dieser Ansatz bereits erfolgreich angewandt, jedoch wird der Interpolationsfaktor α beim Text-Zu-Video Retrieval häufig auf einen festen Wert gesetzt, ohne dass eine systematische Untersuchung der Auswirkungen von α auf die Genauigkeit und Robustheit durchgeführt wird.

Eine weitere Herausforderung ist die Vorverarbeitung und damit die Auswahl der Einzelbilder (Keyframes), die für die Suche verwendet werden, da nicht jedes einzelne Bild eines Videos analysiert werden kann. Die Wahl der Sampling-Strategie beeinflusst sowohl die Suchqualität als auch den Rechen- und Speicheraufwand.

Diese beiden Parameter, die Wahl der Sampling-Strategie und die Wahl des Interpolationsfaktors α, sollen in dieser Arbeit systematisch untersucht werden, um deren Auswirkungen auf die Optimierung der CLIP-basierten Text-zu-Video-Retrieval-Systeme festzustellen.

1.3 Zielsetzung und Aufgabenstellung

Ziel dieser Arbeit ist die Entwicklung und Evaluation eines Python-basierten Prototypen für die Text-zu-Video-Suche. Der Prototyp umfasst die komplette Pipeline von der Vorverarbeitung der Videos über das Finetuning und die Interpolation der Modelle bis hin zur Indexierung und Suche. Außerdem enthält er eine interaktive Webapplikation, welche es ermöglicht, die Modellkonfiguration zu wählen und über Suchanfragen in natürlicher Sprache die Videos zu durchsuchen und die besten Ergebnisse anzuzeigen.

Die Besonderheit dieser Arbeit liegt in der systematischen und empirischen Untersuchung zweier üblicherweise fest gewählter Parameter, unter Berücksichtigung der Begrenzung der verfügbaren Hardware. Hierbei wird, statt α auf einen festen Wert zu setzen, ein Sweep über alle α-Werte, in 0,1er Schritten, durchgeführt und mit vier verschiedenen Sampling-Strategien (festen Bildraten von 0,5, 1 und 2 fps sowie einer TransNetV2-basierten Szenenerkennung) verglichen. Ebenfalls werden zwei unterschiedliche CLIP-Modelle mit jeweils zwei verschiedenen Fine-Tuning-Strategien untersucht und zur Validierung der Ergebnisse ein Out-of-Distribution-Test durchgeführt. Daraus ergeben sich folgende Forschungsfragen:

1. a) Welcher Interpolationsfaktor α zwischen dem Zero-Shot und dem feingetunten Modell liefert die beste Retrieval-Qualität?

   b) Übertrifft ein interpoliertes Modell beide Endpunkte ($\alpha = 0$ und $\alpha = 1$)?

2. Welche Frame-Sampling-Strategie bietet die beste Suchqualität bei vertretbarem Speicheraufwand?

1.4 Aufbau der Arbeit

Die Arbeit ist in sechs Kapitel gegliedert. Kapitel 2 legt die Grundlagen zu Video-Retrieval, Vision-Language-Modellen und erläutert die verwendeten Verfahren und Algorithmen. Außerdem wird die Arbeit in den Stand der Forschung eingeordnet. Kapitel 3 legt die Anforderungen an den Prototypen fest, diskutiert die verschiedenen Möglichkeiten für Frame-Sampling, Modellauswahl und Indexierung und stellt die Architektur des Prototypen vor. Kapitel 4 beschreibt die Implementierung der einzelnen Module sowie den Testaufbau.

Kapitel 5 präsentiert und interpretiert die Ergebnisse und bewertet die beiden Optimierungsstrategien. In Kapitel 6 erfolgt die Zusammenfassung der Arbeit und ein Ausblick auf mögliche Weiterentwicklungen und Untersuchungen.

Kapitel 2: Grundlagen

2.1 Information und Video Retrieval

2.1.1 Grundlagen Information Retrieval

Information Retrieval (IR) beschreibt den Prozess der Suche nach relevanten Informationen in großen Sammlungen unstrukturierter Daten, welche das Informationsbedürfnis des Anfragenden bestmöglich erfüllen [14, S.1–2]. Hierbei stützen sich klassische Suchverfahren auf manuell vergebene Metadaten wie Titel oder Schlagwörter [14, S.17] [9, S.215–216]. Durch das Wachstum von digitalen Medien und damit auch multimedialen Inhalten skaliert dieser Ansatz nicht mehr mit den heutigen Datenmengen. Aus diesem Grund entwickelte sich das Multimedia Information Retrieval (MMIR) als Erweiterung des klassischen Information Retrieval, welches multimediale Daten wie Bilder und Videos gezielt mit einbezieht [9, S.139]. Moderne Systeme extrahieren die Merkmale direkt aus dem Medieninhalt und sind daher nicht mehr auf die aufwändige, manuelle Arbeit von Menschen angewiesen [9, S.19].

Eine der Kernaufgaben des MMIR ist das Ad-hoc Retrieval. Hier spezifizieren die Benutzer einen Informationsbedarf mittels einer Suchanfrage, welche dann ein automatisiertes Verfahren startet und als Ergebnis die zur Suchanfrage relevanten Dokumente beinhaltet [9, S.140–141]. Bei der Verarbeitung dieser Anfragen unterscheidet man zwischen zwei verschiedenen Modellen, den Exact-Match-Modellen und den Best-Match-Modellen.
Bei den Exact-Match-Modellen erfolgt eine binäre Unterteilung der Dokumente in relevant oder nicht relevant. Bei den Best-Match-Modellen wird jedem Dokument ein bestimmter Wert zugeordnet: der Retrieval Status Value (RSV). Dieser Wert repräsentiert die Relevanz zwischen einer Suchanfrage und einem Dokument und ermöglicht somit eine Rangsortierung der Ergebnisse. [9, S.148–158]
Die Known-Item Search (KIS) ist eine spezielle Form des Ad-hoc Retrievals, bei der die Benutzer nach einer ganz spezifischen, ihnen bekannten Entität suchen. Bei dieser Suchaufgabe ist das Best-Match-Ranking essenziell, damit das richtige Ziel, die gesuchte Information, auf den obersten Rangplätzen präsentiert wird.

2.1.2 Grundlagen Video Retrieval

Diese Arbeit beschäftigt sich speziell mit Videos und dem Text-zu-Video Retrieval. Hierbei wird anhand einer textuellen Suchanfrage eine bestimmte Szene in einem Video gesucht.

Dabei ergeben sich mehrere Herausforderungen:

1. Videos bestehen aus einer Abfolge von Bildern über Zeit und Text ist eine Abfolge von Zeichen.

2. Videos bestehen aus einer hohen Anzahl von Einzelbildern, welche innerhalb einer Szene sehr viele visuelle Redundanzen enthalten.

Zu 1: Um eine Vergleichbarkeit für textuelle Suchanfragen zu schaffen, müssen die Textanfragen und Videos in eine gemeinsame Repräsentation überführt werden. Ein gängiger Ansatz ist das Vektorraummodell [9].
Im Vektorraum werden sowohl die Suchanfrage als auch die Multimediadokumente als hochdimensionale Vektoren dargestellt. So werden von einem System inhaltsbasierte Merkmale der Medien extrahiert und in Zahlenvektoren umgewandelt. Je dichter die jeweiligen Vektoren im Raum zusammenliegen, desto höher ist ihre inhaltliche Relevanz zueinander [9, S.152]. Damit kann dann mathematisch die Ähnlichkeit und das RSV, über bestimmte Distanzmaße, wie in dieser Arbeit die Kosinus-Ähnlichkeit (Cosine Similarity), bestimmt werden. Die Kosinus-Ähnlichkeit berechnet den Kosinus des Winkels zwischen den Vektoren [14, S.121-–122]; je näher der Wert an 1 liegt, desto kleiner ist der Winkel zwischen den Vektoren und desto ähnlicher sind sie.

$$\text{Kosinus Ähnlichkeit:} \quad (\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$

Frame-Sampling und Keyframe Extraction

Zu 2: Um die Redundanz innerhalb eines Videos zu reduzieren, wird nicht jedes einzelne Bild eines Videos mit einer Texteingabe verglichen, sondern es wird versucht mit möglichst wenig Bildern das Video so gut wie möglich zu repräsentieren. Diese Bilder nennt man Keyframes. Die dazu existierenden Strategien werden in Abschnitt 2.3.1 erläutert.

2.2 Neuronale Netze und Vision Language Modelle

2.2.1 Künstliche Neuronale Netzwerke

Grundlagen und Aufbau

Künstliche Neuronale Netze (KNN) sind historisch von dem Aufbau menschlicher Gehirne inspiriert. Im menschlichen Gehirn leiten Neuronen elektrische Impulse über Synapsen weiter. Diese Verbindungsstellen können durch bestimmtes Lernen verstärkt werden. Dieses Prinzip wird bei künstlichen Neuronalen Netzen mathematisch simuliert, um Muster in komplexen Daten zu finden [4, S.1-–3]. Die fundamentale Recheneinheit eines KNN ist das

künstliche Neuron. Dieses verarbeitet Eingaben in einem zweistufigen Prozess.

1. **Die Voraktivierung:** Hier empfängt das künstliche Neuron N Eingabewerte $(x_1, \ldots, x_N)$ und multipliziert diese mit Gewichtsfaktoren $(w_1, \ldots, w_N)$, welche die Stärke der jeweiligen Verbindung repräsentieren. Addiert wird noch ein Schwellenwert (Bias). Alles zusammen ergibt die Voraktivierung a des Neurons.

$$a = \sum_{i=1}^{N} w_i x_i + b$$

2. **Die Aktivierung:** Der Voraktivierungswert a wird anschließend durch eine nichtlineare Aktivierungsfunktion $h(a)$ transformiert. Daraus ergibt sich der endgültige Ausgangswert z des Neurons.

$$z = h(a)$$

Ohne die Aktivierungsfunktion würde das gesamte Netzwerk mathematisch nur eine lineare Funktion darstellen und wäre nicht in der Lage komplexe, nicht lineare Zusammenhänge zu lernen.

Mehrere Neuronen werden in aufeinanderfolgenden Schichten (Layern) organisiert. Am Anfang des Netzes befindet sich die Eingabeschicht (Input Layer), welche die Rohdaten wie zum Beispiel die Pixelwerte eines Bildes empfängt. Danach folgen die verborgenen Schichten (Hidden Layers), wobei jede Schicht die Ausgabe der vorherigen Schicht verarbeitet. Am Ende des Netzes befindet sich die Ausgabeschicht (Output Layer), welche die endgültige Ausgabe liefert [1, Kap.1]. Dieses System ermöglicht dem KNN, ein hierarchisches Verständnis der Daten zu entwickeln, wobei die unteren Schichten einfache Merkmale erkennen und die höheren Schichten komplexere Muster identifizieren.

Training neuronaler Netze

Damit ein Neuronales Netzwerk gute Ergebnisse liefert, müssen die Gewichte und Bias-Werte der einzelnen Neuronen angepasst werden. Zur Vorbereitung werden die verfügbaren Daten in Trainings-, Validierungs- und Testdaten aufgeteilt. Das Training des Netzwerkes erfolgt auf den Trainingsdaten, während die Validierungsdaten zur Überprüfung der Leistung während des Trainings verwendet werden. Zu Beginn des Trainings werden hierzu die Parameter zufällig initialisiert. Das Training erfolgt zyklisch in sogenannten Epochen (vollständige Durchläufe durch die Trainingsdaten) und lässt sich in drei Einzelteile zerlegen [1, Kap.1]:

1. **Fehlerfunktion (Loss Function):** Diese bestimmt mathematisch die Abweichung zwischen der Vorhersage des Netzwerkes und dem tatsächlichen Zielwert [1, S.11].

2. **Backpropagation Algorithmus:** Um die Gewichte und Bias-Werte anzupassen, fließt der berechnete Fehlerwert der Fehlerfunktion rückwärts durch das Netzwerk und bestimmt wie stark jedes einzelne Gewicht und Bias zum Gesamtfehler beigetragen haben [1, S.21–22].

3. **Stochastischer Gradientenabstieg (SGD):** Dieser Optimierungsalgorithmus passt die Gewichte und Bias-Werte schrittweise entgegen der Fehlerrichtung an. Die Schrittweite wird durch einen Parameter namens Lernrate bestimmt [1, S.13].

Mit diesem Verfahren wird das Netzwerk über viele Zyklen hinweg trainiert. In jedem Zyklus werden die Parameter minimal angepasst, um den Fehler zu minimieren. Das Ziel des Trainings ist, das Netzwerk aus den Trainingsdaten so viele Muster lernen zu lassen, sodass es andere, bisher ungesehene Daten erkennen kann. Die Fähigkeit, Dinge zu erkennen, die dem Netzwerk zuvor unbekannt waren, nennt man Generalisierung [1, S.169].

Ein für das Verständnis dieser Arbeit wichtiger Parameter beim Trainieren ist das Warmup. Hierbei wird die Lernrate zu Beginn des Trainings langsam erhöht, um das Netzwerk an die Daten zu gewöhnen und eine stabile Annäherung an das Optimum zu gewährleisten [6]. Zudem sollte die Batchsize erwähnt werden, die die Anzahl der vom Modell gesehenen Trainingsbeispiele beschreibt, bevor dessen Gewichte angepasst werden.

Auf SGD aufbauende adaptive Verfahren wie Adam bzw. AdamW passen die Schrittweite pro Parameter anhand gleitender Mittel der Gradienten an und werden oft bei Transformer-Architekturen verwendet. In dieser Arbeit wird der AdamW Optimierungsalgorithmus in der Standardkonfiguration des OpenCLIP-Trainingsmoduls verwendet (vgl. 4.3.2).

Wird ein Netzwerk zu lange trainiert, besteht die Möglichkeit, dass das Netzwerk die Trainingsdaten zu gut lernt und sich die Gewichte zu stark an die Trainingsdaten anpassen. Dies ist daran erkennbar, indem die Leistung auf den Trainingsdaten weiterhin steigt, während die Leistung auf den Validierungsdaten abnimmt. Dieses Phänomen nennt man Overfitting.

2.2.2 Visuelle Feature-Extraktion

Um visuelle Inhalte zu analysieren und bestimmte Merkmale (Features) zu extrahieren, existieren in der Computer Vision verschiedene Ansätze. Klassische Faltungsnetzwerke (Convolutional Neural Networks, CNNs) basieren auf der Annahme, dass benachbarte Pixel in einem Bild oft zusammenhängende Informationen repräsentieren. Durch die Anwendung von Filtermasken (Kernels), welche schrittweise über das gesamte Bild geschoben werden, werden lokale Merkmale wie zum Beispiel Kanten erkannt [3, S.292–293]. In tieferen Schichten des Netzwerkes werden dann komplexere Merkmale wie Formen und Objekte erkannt [1, S.331–332].

Eine moderne Alternative zu den CNNs sind die Vision Transformer (ViT) [6]. Diese sind inspiriert von klassischen Transformer Netzwerken, die ursprünglich zur Verarbeitung von Text entwickelt wurden.

Die ViT betrachten nicht jeden Pixel einzeln, sondern teilen das Bild in quadratische Bildausschnitte (Patches) auf, da aufgrund der Selbstaufmerksamkeit die Beziehung zwischen allen Paaren von Eingaben berechnet wird und sonst der Rechenaufwand zu groß wäre. Ein Bild, das 224×224 Pixel groß ist, wird dadurch in ein Raster aus 14×14 Patches zerlegt [6].

Jedes dieser Patches wird als nächstes flachgedrückt (Flattening), sodass ein Vektor entsteht, der alle Pixelwerte (inklusive der RGB Werte) des Patches enthält. Diese Vektoren werden dann durch eine lineare Projektion auf eine festgelegte Dimension D abgebildet (Patch-Embeddings). Am Anfang der Sequenz von Patch Embeddings wird zusätzlich ein sogenanntes `[class]`-Token hinzugefügt. Dieses enthält am Anfang keine Informationen über das Bild. Außerdem wird jedem Token ein gelernter Positionsvektor (Position Embedding) hinzuaddiert, um Informationen über die räumliche Anordnung der Patches zu erhalten [6].

Die vorbereitete Sequenz an Tokens L durchläuft daraufhin strukturell identische Schichten (Transformer Layer). In jeder Schicht wird für jedes Token die Beziehung zu allen anderen Tokens berechnet [4]. Diesen Mechanismus nennt man Selbstaufmerksamkeit (Self-Attention). Die Ausgabe jeder Schicht wird dann als Eingabe für die nächste Schicht verwendet. Nach mehreren Schichten enthält das `[class]`-Token Informationen über das gesamte Bild, da es in jeder Schicht die Beziehungen zu allen anderen Tokens berechnet hat. Am Ende dient das `[class]`-Token als Merkmalsvektor für das gesamte Bild und die anderen Tokens können verworfen werden. So lernt das Modell eine vollständige Repräsentation des Bildes in diesem Token abzubilden. Abbildung 2.1 veranschaulicht den Aufbau.

In dem beschriebenem Vorgang liegt der Vorteil der Vision Transformer Modelle gegenüber den klassischen CNNs. Denn während CNNs nur schrittweise lokal über Filtermasken Merkmale erkennen, verfügt der Transformer schon von der ersten Schicht an über den vollständigen Bildkontext und kann somit direkt globale Merkmale erkennen.

2.2.3 Vision Language Modelle

Klassische Computervision Modelle kategorisieren Bilder anhand von einem vorgefertigten Set verschiedener Klassen [16]. Um eine sprachbasierte Videosuche zu ermöglichen wird eine Form von Modell, die visuelle und textuelle Informationen in einem einzigen Modell integriert, benötigt. Diese Art von Modellen werden Vision Language Modelle genannt.
Es gibt zwei verschiedene Architekturen, die Single- und die Dual-Stream Architektur [2]. Single Stream Architekturen mischen Bild- und Textdaten bereits am Anfang des Modells miteinander und verhindern damit das Vorberechnen der Bildrepräsentationen.
Im Gegensatz dazu verarbeitet die Dual-Stream Architektur beide Modalitäten getrennt:

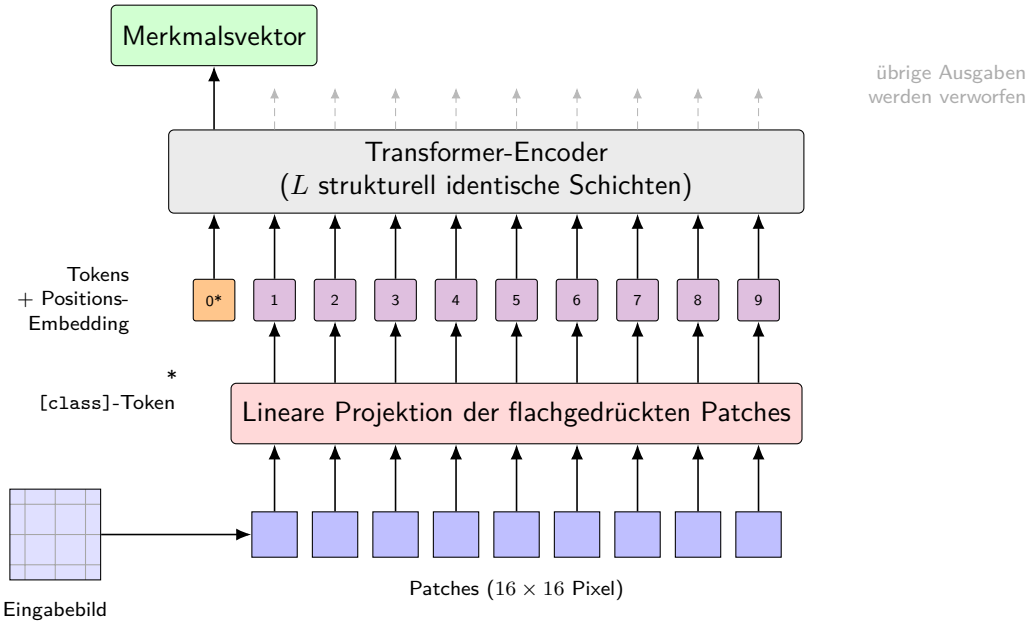


Abbildung 2.1: Aufbau des Vision Transformer. Eigene Darstellung in Anlehnung an [6].

- Der Bild-Encoder (zB. ein Vision-Transformer) übersetzt ein Bild in einen Vektor.
- Der Text-Encoder (ein klassischer Text-Transformer) übersetzt eine Beschreibung in einen Vektor.

Beide oben genannten Vektoren werden in einem gemeinsamen Vektorraum (Joint Embedding Space) projiziert. Das Training eines solchen Netzwerks folgt dem Prinzip des kontrastiven Lernens (Contrastive Learning) [4, S.191–193]. Hierbei versucht das Modell nicht die Bildunterschrift vorherzusagen, sondern wird darauf trainiert, mit einem Batch von N Bild-Text-Paaren die Kosinus-Ähnlichkeit der N zusammengehörigen Paare zu maximieren und die Ähnlichkeit der $N^2 - N$ unpassenden Paare zu minimieren.

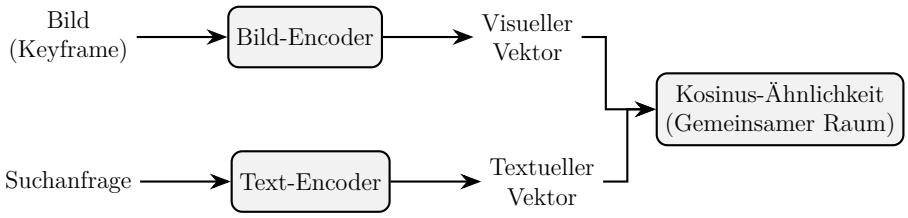


Abbildung 2.2: Übersicht des gemeinsamen Vektorraums

2.2.4 (Open-)CLIP

Ein bahnbrechendes Modell der Vision Language Modelle ist CLIP (Contrastive Language-Image Pre-training) [16], ein von OpenAI 2021 vorgestelltes Dual-Stream Modell, das auf rund 400 Millionen Bild-Text-Paaren trainiert wurde. Aufgrund dieser Datenmenge konnte

das Modell ein universelles Verständnis für visuelle Konzepte in Verbindung mit natürlicher Sprache entwickeln und kann somit auf völlig neue Daten angewandt werden, ohne ein erneutes Training durchzuführen. Diese Fähigkeit wird als Zero-Shot-Fähigkeit bezeichnet und ist eine der herausragenden Fähigkeiten von CLIP.

Da jedoch der gigantische Trainingsdatensatz von CLIP nicht veröffentlicht wurde, entstand in der Open-Source-Gemeinschaft das Projekt OpenCLIP [11]. Dieses Projekt repliziert die Architektur von OpenAIs CLIP und wurde auf öffentlich zugänglichen Datensätzen wie dem LAION-400M oder dem LAION-5B trainiert. Einige dieser offenen Modelle übertreffen die Robustheit und Genauigkeit der originalen Modelle [5].

2.3 Verwendete Algorithmen und Verfahren

2.3.1 Frame-Sampling und Keyframe Extraktion

Zur näheren Betrachtung der Herausforderung der Redundanzen innerhalb eines Videos, wird die Ausgangslage erneut dargestellt. Existiert ein Video mit einer Länge von 10 Sekunden und einer Bildrate von 30 fps, ergäbe das 300 Frames pro Video. Diese 300 Frames müssten, um sie mit einer textuellen Suchanfrage vergleichen zu können, in Vektoren umgewandelt werden. Das würde die Berechnung der Vektoren für jedes Bild beim Preprocessing und die Index-Größe und damit Suchzeit in die Höhe treiben. Allerdings sollte auch darauf geachtet werden, dass nicht zu wenige Frames verwendet werden, da sonst relevante Szenen gar nicht erst gefunden werden können. Aus diesem Grund ist es wichtig, eine Balance zwischen Redundanz und Informationsverlust zu finden [9, 13].

Feste Abtastraten

Das einfachste Prinzip ist es, feste Abtastraten zu verwenden. Es würde jeder n-te Frame unabhängig vom Inhalt verwendet werden. Vorteile sind hierbei zum Beispiel die Simplizität und die einfache Skalierbarkeit. Da es jedoch unabhängig vom Inhalt funktioniert, liegt gerade hierin die Schwäche der Methode. So können bei längeren, gleichbleibenden Szenen viele Redundanzen entstehen und bei sehr kurzen Szenen kann die wichtige Stelle übersprungen werden.
Dieses Vorgehen ist trotz der Nachteile sehr beliebt in der Literatur und wird von vielen Arbeiten, auch im Kontext des VBS, verwendet z.B. [13, 10].

Inhaltsbasiertes Sampling und TransNetV2

Eine etwas aufwändigere Methode ist das Inhaltsbasierte Sampling. Hierbei wird sich zum Vorteil genommen, dass Videos meist in einzelne Shots (ununterbrochene Kameraaufnahmen) gegliedert sind. Es wird also versucht ein Keyframe pro Shot zu extrahieren und

somit eine Repräsentation ohne Informationsverlust und Redundanz zu haben. Allerdings ist es nicht immer trivial einzelne Shots zu trennen. So können harte Schnitte zwar noch verhältnismäßig leicht erkannt werden, aber gerade weiche Übergänge sind schwer von beispielsweise Beleuchtungswechseln zu trennen [21]. Ein Lösungsansatz zur Erkennung von einzelnen Shots ist TransNetV2 [23]. Dabei handelt es sich um ein Neuronales Netz, welches eine Sequenz von Frames gleichzeitig verarbeitet und als Ausgabe eine Wahrscheinlichkeit pro Frame für einen Shot-Übergang liefert. Trainiert wurde das Netz auf synthetisch erzeugten Übergängen und evaluiert auf Datensätzen wie ClipShots oder BBC Planet Earth.

2.3.2 Von Frame- zu Video-Ähnlichkeit

Nach der Extraktion der Keyframes aus den Videos, liegen mehrere Vektoren pro Video vor, die jeweils die Inhalte der einzelnen Keyframes repräsentieren. Da beim Video-Retrieval ein Ranking der ähnlichsten Videos gesucht wird, muss pro Video ein einzelner Ähnlichkeitswert bestimmt werden. Die Aufgabe besteht nun darin, den „richtigen“ Vektor auszuwählen. Das geschieht meist über folgende Verfahren:

Ansatz 1: Mean Pooling

Beim Mean Pooling werden aus den extrahierten Frame Vektoren, ein einziger Videovektor gemittelt. Dies verringert die Anzahl an Vektoren und erleichtert das Ermitteln des richtigen Videos. Der Nachteil ist, dass sich bei längeren Videos oder Videos mit unterschiedlichen Szenen die Inhalte vermischen, was dazu führen kann, dass besonders kleine aber vielleicht wichtige Abschnitte „verwaschen“ [13].

Ansatz 2: Max-Similarity

Bei der Max-Similarity bleibt jeder Keyframe erhalten. Die Suche wird über alle Keyframes durchgeführt und der Keyframe mit der größten Ähnlichkeit zum Text, wird für das spezifische Video ausgewählt, wie nach dem Prinzip „der beste Frame gewinnt“.

Da bei KIS eine ganz bestimmte Szene gesucht wird und nicht das Video im Mittel, wird für die Suche vorwiegend der Max-Similarity Ansatz gewählt, um den besten Frame zu finden.

2.3.3 Modell-Interpolation

Soll ein Modell für eine bestimmte Aufgabe oder Domäne spezialisiert eingesetzt werden, bietet sich das Fine-Tuning oder Anpassen des Modells an. Hierbei wird das Modell nochmal mit einem vergleichsweise kleinen, anwendungsspezifischen Datensatz trainiert. Dazu muss

zwischen dem teilweisen oder vollständigen Fine-Tuning ausgewählt werden, bei denen entweder nur das Ende des Netzwerks oder das gesamte Netzwerk angepasst wird. Der Vorteil ist, dass mit wenig Zeitaufwand sehr gute Ergebnisse in der Zielaufgabe oder Zieldomäne erzielt werden. Dabei besteht die Gefahr, dass es zu einer Überanpassung (Overfitting) kommen kann, was zum Verlust der Verallgemeinerungsfähigkeit oder Robustheit des Modells führt.
Insbesondere besteht die Gefahr des Verlusts bei CLIP Modellen, die beim Fine-Tuning schnell ihre Zero-Shot-Fähigkeiten verlieren. Zur Verbesserung dieser Schwachstelle wurde das WiSE-FT (Weight-Space Ensembles for Fine-Tuning) Verfahren entwickelt [27].

Bei diesem Verfahren werden, anstatt aufwändig die Ergebnisse von zwei separaten Modellen zu mitteln, die internen Parameter (Gewichte) der Modelle über eine lineare Interpolation fusioniert:

$$\theta_{\text{WiSE-FT}} = (1 - \alpha) \cdot \theta_0 + \alpha \cdot \theta_1$$

- θ_0 stellt die Gewichte des Zero-Shot-Basismodells dar.
- θ_1 stellt die Gewichte desselben Modells dar, welches auf eine bestimmte Domäne feingetunt wurde.
- α ist der Interpolationskoeffizient, der das Gleichgewicht steuert.

Dieses Verfahren setzt eine identische Architektur, sowie eine gemeinsame Initialisierung voraus. Als Ergebnis entsteht ein neues Modell, welches die Domäne, für die es angepasst wurde, sehr gut erkennt und dabei seine Zero-Shot-Fähigkeit nicht einbüßt. Es entstehen keine zusätzlichen Rechen- oder Speicherkosten [27].

2.3.4 Effiziente Vektorsuche

Nachdem die Keyframes der Videos durch das neuronale Netz verarbeitet wurden, liegen diese als multidimensionale Feature-Vektoren vor. Bei einer Suchanfrage müssen nun die vorberechneten Keyframe-Vektoren, mit den Vektoren einer textuellen Suchanfrage verglichen werden. Da dieser Vergleich bei riesigen Datensätzen sehr rechenintensiv ist, werden hierfür häufig approximative Suchverfahren verwendet. Bei moderat großen Datensätzen ist es jedoch von Vorteil, eine exakte Suche durchzuführen. Diese ermittelt garantiert die tatsächlichen nächsten Nachbarn und vermeidet damit, im Gegensatz zu approximativen Verfahren, den Verlust an Suchgenauigkeit [7].

Um diesen Vektorvergleich umzusetzen, wird die Programmbibliothek FAISS (Facebook AI Research Similarity Search) verwendet [7]. FAISS bietet eine Sammlung von Indextypen für die Ähnlichkeitssuche und teilt diese in 2 verschiedene Gruppen:

- Flat-Indizes, welche die Vektoren unkomprimiert speichern und jeden einzelnen vergleichen

- Approximative Indizes, welche durch unterschiedliche Verfahren wie Nachbarschaftsgraphen nur einen Teil der Vektoren prüfen

Die approximativen Verfahren übertreffen die Flat-Indizes im Geschwindigkeits- und Speicheraspekt, verlieren aber im Bereich Genauigkeit.

2.4 Verwandte Arbeiten

2.4.1 VBS

Den Anwendungskontext dieser Arbeit bildet der Video Browser Showdown (VBS) [25]. Dies ist ein jährlich stattfindender internationaler Videowettbewerb, bei dem die Effizienz der teilnehmenden Tools zur Videosuche anhand eines gemeinsamen Datensatzes vor Publikum bewertet und demonstriert wird. Ein Teil dieses Wettbewerbs ist die Textual Known Item Search, bei dem die Teilnehmer und Teilnehmerinnen anhand einer textuellen Beschreibung einer Szene diese suchen und finden müssen. Das Verfahren der Suche zu optimieren ist das zentrale Anliegen dieser Arbeit.

diveXplore

Das System diveXplore [19] ist ein erfolgreicher Teilnehmer des Video Browser Showdowns und ist eine Inspirationsquelle dieser Arbeit. Zum Extrahieren der Keyframes setzt dieses System TransNetV2 ein und nutzt ein OpenCLIP-Modell zum Extrahieren der Merkmalsvektoren. Diese werden mithilfe einer FAISS Ähnlichkeitssuche indiziert.

2.4.2 WiSE-FT

Wie in Abschnitt 2.3.3 angeführt, wird in der Arbeit *Robust fine-tuning of zero-shot models* [27] die WiSE-FT Methode entwickelt. Diese Methode soll das Problem des Robustheitsverlusts bei Out-Of-Distribution(OOD) Daten beim Trainieren von CLIP Modellen beheben. Out of Distribution Daten sind Daten, die nicht in der Trainingsmenge enthalten waren und somit für das Modell unbekannt sind. Ergänzend wurde in dieser Studie auch ermittelt, dass neben der Steigerung Robustheit in anderen OOD Datensätzen, auch die In-Distribution (Daten innerhalb der Trainingsdaten) Tests eine bessere Genauigkeit als das rein feingetunte Modell bieten.

2.4.3 Open-VCLIP

Das Open-VCLIP Paper [26] passt CLIP auf Zero-Shot Video Erkennung an und nutzt dabei die Gewichtsinterpolation als Methode, um die Generalisierungsfähigkeit beizubehalten. Dabei wird die Gewichtsinterpolation nicht wie bei WiSE-FT auf Bilder sondern auf Videos angewendet.

2.4.4 CLIP-Optimization for Video Search

Ueki et al. untersuchen in einer kürzlich veröffentlichten Arbeit [24] Optimierungsstrategien für Videosuche unter Anwendung von OpenCLIP-Modellen. Der Fokus liegt hierbei auf der Präzision des Modells in der Zieldomäne, ohne die Generalisierungsfähigkeit einzubüßen. Beim Trainieren wurde eine angepasste Version des LIT-Verfahrens [29] angewandt, hierbei wird beim Trainieren des CLIP Modells der Image-Encoder eingefroren und nur der Text-Encoder trainiert. Ueki et al. verändern dieses Verfahren, indem sie nur den Text-Encoder und die letzten Layer des Image-Encoders trainieren. Das fertig trainierte Modell wird daraufhin mit der WiSE-FT Methode mit dem Zero-Shot-Modell 1:1 gemischt. Zum Testen wurde jeder zehnte Frame extrahiert und nach dem Max-Similarity-Prinzip ausgewählt. Die vorgeschlagenen Strategien wurden an 27 Modellen getestet und zeigten bei allen Modellen eine höhere Retrieval Accuracy.

Der Interpolationsfaktor α wird fest auf 1:1($\alpha = 0,5$) gesetzt, ohne ihn zu untersuchen. Ebenso wenig wird die Frame-Sampling-Strategie unter die Lupe genommen. Eine systematische Untersuchung des Interpolationsfaktors, insbesondere in Kombination mit unterschiedlichen Frame-Strategien, liegt bislang nicht vor.

Kapitel 3: Konzeption

3.1 Anforderungen und Zielstellung

3.1.1 Zielstellung und Forschungsfragen

Diese Arbeit hat die Zielstellung sich mit der Optimierung von CLIP-basierten Text-zu-Video Retrieval Systemen auseinanderzusetzen. Im vorherigen Abschnitt 2.4.4 wurde dargestellt, dass besonders zwei Parameter bei der Optimierung von Text-zu-Video Retrieval Systemen, der Interpolationsfaktor beim Modell und die Frame-Sampling-Strategie, meist fest gewählt und nicht empirisch untersucht werden. Ziel dieser Arbeit ist diese beiden Faktoren, mithilfe eines Python-basierten Prototypen, für die Text-zu-Video Suche systematisch zu untersuchen.
Aus dieser Zielstellung ergeben sich folgende Forschungsfragen:

1. a) Welcher Interpolationsfaktor α zwischen dem Zero-Shot und dem feingetunten Modell liefert die beste Retrieval-Qualität?

   b) Übertrifft ein interpoliertes Modell beide Endpunkte ($\alpha = 0$ und $\alpha = 1$)?

2. Welche Frame-Sampling-Strategie bietet die beste Suchqualität bei vertretbarem Speicheraufwand?

Die Optimierung des Modells und damit des Interpolationsfaktors befasst sich genauer mit der Frage, wie das Modell bestmöglich angepasst werden kann, ohne zu viel Robustheit einzubüßen. Dabei wird der Fokus auf die in Abschnitt 2.3.3 vorgestellte Interpolation von einem feingetunten und einem Zero-Shot CLIP Modell und genauer dem Faktor α gelegt.

Die Optimierung der Frame-Sampling Strategie soll sich genauer mit dem Unterschied zwischen der in Abschnitt 2.3.1 präsentierten Inhaltsbasierten- und Festen-Frame-Strategie befassen und welche davon die besseren Ergebnisse liefert, ohne zu viel Speicherplatz einzunehmen. Dafür werden im Folgenden vier verschiedene Strategien ausgewählt und untersucht.

3.1.2 Funktionale Anforderungen

Die Beantwortung dieser Forschungsfragen erfordert die Festlegung einiger funktionalen Anforderungen an das zu entwickelnde System. Diese werden folgend in vier verschiedenen Kategorien unterteilt.

Vorverarbeitung

Das System muss in der Lage sein aus Videodaten Einzelbilder (Keyframes) extrahieren zu können. Zur Untersuchung muss das System verschiedene Frame-Sampling Strategien unterstützen, sowohl inhaltsbasierte Szenenerkennung als auch feste Abtastraten. Dabei muss beachtet werden, dass die Frames eindeutig den einzelnen Videos zugeordnet werden können, damit die nachfolgende Evaluation fehlerfrei verläuft.

Modell und Fine-Tuning

Zur Verarbeitung muss ein vortrainiertes CLIP-Modell geladen und mithilfe des Datensatzes feingetunt werden. Beim Fine-Tuning muss das System die Trainings-Checkpoints je Epoche speichern, damit im Nachhinein der beste Checkpoint zum Testen verwendet werden kann. Des Weiteren muss das System das vortrainierte und das feingetunte Modell per Gewichtsinterpolation (WiSE-FT) zu einem neuen Modell mischen. Der Mischfaktor α muss leicht anpassbar sein, um seine Untersuchung zu ermöglichen.

Indexierung und Suche

Zur Indexierung und damit der Ermöglichung der Suche muss das System alle extrahierten Keyframes in Embedding-Vektoren umwandeln. Aus den Embeddings muss das System einen durchsuchbaren Vektorindex aufbauen. Auf der anderen Seite muss eine Texteingabe entgegengenommen und ebenfalls in einen Embedding-Vektor umgewandelt werden. Der aus dieser Suche entstandene Vektor muss möglichst schnell mit den anderen Keyframe Vektoren verglichen werden, um die ähnlichsten Frames zu finden und nach Relevanz zu sortieren. Bei der Relevanz muss darauf geachtet werden, dass pro Video nur der beste Treffer gezählt wird (Max-Similarity-Prinzip, siehe Abschnitt 2.3.2).

Evaluation

Zum Vergleich aller Ergebnisse, muss das System die Retrieval-Qualität der verschiedenen Ansätze zuverlässig und konsistent messen. Dafür müssen über alle Tests hinweg mehrere Metriken bestimmt werden. Das System muss außerdem diese Evaluation automatisch über verschiedene Modell- und Strategie-Kombinationen ausführen, um händische Fehler zu minimieren.

Veranschaulichung

Zur Veranschaulichung sollte eine Benutzeroberfläche bereitgestellt werden, über die eine Text-zu-Video Suche durchgeführt werden kann. Dabei sollten sämtliche Variablen (Modellauswahl und Frame-Strategie) frei einstellbar sein und die mit der Suchanfrage ermittelten Bilder mit Videonamen und Ähnlichkeitswerten angezeigt werden.

3.1.3 Nicht-funktionale Anforderungen

Bei dem Entwurf des Systems müssen Anforderungen beachtet werden, die die Qualität und den Rahmen des Systems sicherstellen. Zuerst muss das System auf begrenzter Hardware lauffähig sein. Für die Untersuchung steht ein Consumer-Rechner mit einer GTX 1080 Grafikkarte von NVIDIA aus dem Jahr 2016 mit 8GB Grafikspeicher zur Verfügung. Diese Einschränkung ist zu erwähnen, da diese viele Designentscheidungen beeinflusst. Außerdem soll die Suchanfrage bei wachsender Anzahl von Frames, in interaktiver Zeit beantwortet werden. Dies garantiert eine schnelle Suche, die im Kontext des VBS erwünscht ist. Hinzuzufügen ist die Pflicht bei einer empirischen Arbeit die Experimentbedingungen vergleichbar zu halten und während der Experimente so wenig Veränderungen am System vorzunehmen wie möglich, um die Vergleichbarkeit der Ergebnisse nicht zu gefährden. Für nachfolgende Untersuchungen und Tests müssen die Ergebnisse einfach reproduzierbar und erweiterbar sein. Somit sollte gewährleistet sein, dass andere Modelle, Sampling-Strategien und Datensätze unabhängig voneinander austauschbar sind.

3.2 Evaluationsdesign

3.2.1 Datensatzwahl

Der gewählte Datensatz ist der MSR-VTT [28] aus dem Jahr 2016. Dieser besteht aus 10.000 Internet Video-Clips mit einer Gesamtlänge von 41,2 Stunden und circa 20 Captions (Videobeschreibungen) je Video. Die Durchschnittslänge der Videos beträgt circa 15 Sekunden. Die Videos zeigen diverse Szenen und reichen von TV-Show, Film und Cartoon Clips, über Musik-, Sport- und Kochvideos zu einfachen Naturaufnahmen oder Schmink-Tutorials. Für diese Aufteilung wurden 257 beliebte Suchanfragen an eine Videosuchmaschine gestellt und jeweils 118 der resultierenden Videos erfasst. Aus diesen Videos wurden circa 30.000 Clips erfasst und davon 10.000 zufällig ausgewählt [28]. Die circa 20 Captions pro Video wurden von Menschen erstellt. Aufgrund der großen Diversität an verschiedenen Videos und der menschlichen Captions, passt der MSR-VTT sehr gut zum Kontext des textual KIS und dem Setting des VBS. Außerdem ist er durch seine Größe von circa 6,4 GB gut auf einem eigenen Rechner zu verarbeiten; Ausführliche Beschreibung der Verarbeitung in Abschnitt 3.2.3. Abbildung 3.1 zeigt vier Beispielvideos mit jeweils fünf Captions aus dem MSR-VTT-Datensatz.

Zum Trainieren wurde der offizielle Train-, Validation- und Testsplit verwendet. Der Trainingsdatensatz umfasst die ersten 6513 Videos (0-6512), mit diesem wurde das Fine-Tuning ausgeführt. Der Val-Split umfasst weitere 497(6513-7009) Videos und wurde zum Validieren des Modells während des Trainings verwendet. Auf dem Test Split, betreffend die Videos 7010 bis 9999, wurden die Tests durchgeführt. Ein Teil der Literatur nutzt andere Splits wie den 1K-A, welcher einen kleineren Testsplit hat. Folglich sind die absoluten Recall-Werte nur eingeschränkt mit den Systemen, die andere Splits nutzen vergleichbar.

Zur Prüfung, ob Gewichtsinterpolation unter Domänenverschiebung stärker wirkt, wird ein Out-Of-Distribution-Test durchgeführt. Dabei wird, abgesehen von den MSR-VTT-Daten, der Charades-Datensatz [20] aus dem Jahr 2016 herangezogen. Dieser besteht aus 9.848 Videos mit einer Durchschnittslänge von 30 Sekunden und 27.847 Captions, also ca. 2,8 pro Video. Die Videos zeigen 267 Personen, die alltägliche Situationen in ihrem Zuhause nachspielen. Zur Evaluation wurde der Test-Split genutzt, der 1863 Videos mit 4537 Captions beinhaltet.

Der Domänenunterschied zwischen beiden Datensätzen lässt sich an zwei Punkten festmachen. Zum einen sind die Captions der Videos mit einer Durchschnittslänge von 23,1 Wörtern pro Caption deutlich länger als die Captions des MSR-VTT-Datensatzes, die im Durchschnitt aus 9,3 Wörtern pro Caption bestehen. Zum anderen decken die Videos des MSR-VTT-Datensatzes eine große Bandbreite an Themen ab, während die Videos des Charades-Datensatzes nur in Innenräumen nachgestellte Alltagshandlungen zeigen. Abbildung 3.2 zeigt vier Beispiele für den Charades-Datensatz.

3.2.2 Metriken

Zur Bewertung der Ergebnisse wurden folgende Metriken berücksichtigt:

- **Recall@K:** Beschreibt den Anteil der Anfragen, bei denen das richtige Video unter den Top-K Ergebnissen liegt.

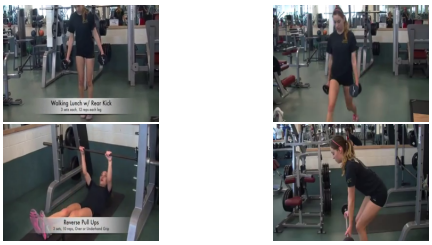
$$\text{Recall@K} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\text{rank}_i \leq K]$$

- **Median-Rank:** Gibt den Medianrang des korrekten Videos über alle Anfragen wieder.

$$\text{Median}(R) = \begin{cases} r_{\frac{N+1}{2}} & \text{wenn } N \text{ ungerade} \\ \frac{1}{2}\left(r_{\frac{N}{2}} + r_{\frac{N}{2}+1}\right) & \text{wenn } N \text{ gerade} \end{cases}$$

- **Mean-Rank:** Der Mean-Rank ist der Durchschnittsrang des korrekten Videos über alle Anfragen.

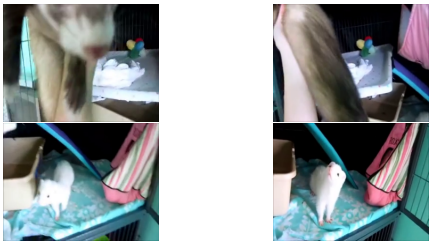
$$\text{Mean-Rank} = \frac{1}{N} \sum_{i=1}^{N} r_i$$



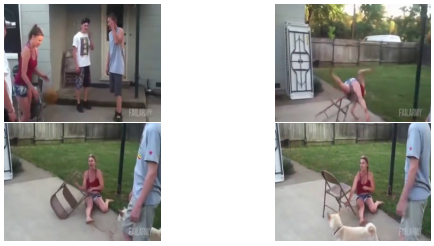
1. there is a woman in black is work out in a jim
2. a black color dressed girl is doing exercise
3. in the gym a teen girl wearing a black t-shirt and a black short is exercising she does push up on the ground
4. a woman is in a gym doing different kinds of exercises
5. a woman in black doing exercises in gym



1. a guy is doing tricks on a bike
2. a man does bike tricks
3. a man doing some adventure with cycle
4. a man doing tricks on a bicycle
5. a man doing tricks on a bike



1. a ferret is held by his owner
2. a ferret is watching the filmer
3. a ferret stands on a pad
4. a man cooking food
5. a man playing with rats



1. a girl falls over a chair when she tries a basketball trick shot
2. there is a woman in red is falling down while playing
3. there is a woman in red is talking with a man
4. a girl in a red shirt and blue shorts with a basketball trying to use a folding chair and a spring board to make a shot
5. a girl tries to make a basketball shot and falls painfully over a chair

Abbildung 3.1: Vier Beispielvideos aus dem MSR-VTT-Datensatz, mit jeweils fünf zugehörigen Captions.

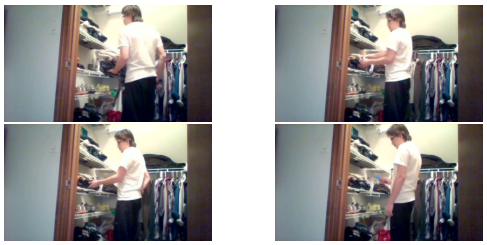
Im Kontext des VBS und der textuellen KIS ist der Einsatz zwei dieser Metriken besonders sinnvoll. Zum einen wurde Recall@K ausgewählt, da im Kontext des Wettbewerbes, mit dem Anlass das richtige Video anhand einer vorgegebenen Suchanfrage zu finden, der Fokus auf die ersten K Videos gesetzt wird. Daraus folgt, dass dieser Wert besonders aussagekräftig ist. Gewählt wurden für den Parameter K die Werte $K \in \{1, 5, 10\}$. Zum anderen wurde der Median-Rank gewählt, da dieser keine so große Anfälligkeit gegenüber Ausreißern besitzt. Bei einem Testsplit von knapp 3000 Videos würde ein Ausreißer den gesamten Durchschnitt beeinflussen, indem er diesen nach oben zieht. Trotz dieser Schwachstelle wird der Mean-Rank als ergänzende Vergleichsmetrik mit aufgenommen.

3.2.3 Evaluationsablauf

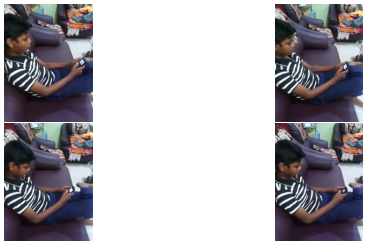
Da im VBS bei den textuellen KIS Aufgaben das System die richtigen Videos anhand eines vorgegebenen Textes finden muss, wird auch diese Retrieval Richtung von Text zu Video gewählt. Für die knapp 3000 Testvideos wurden per festem Zufallsseed, um Reproduzierbarkeit zu gewährleisten, jeweils eine der Captions pro Video extrahiert, die den Aufgabentext des Wettbewerbs simuliert. Daraufhin wird die Caption encodiert und gegen die gesamten Framevektoren verglichen. Da jedoch pro Video mehrere Keyframes



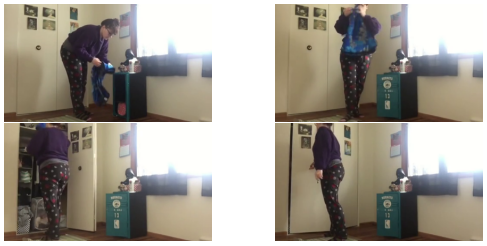
1. A person holds a camera up to a light on the ceiling
2. A person in a bedroom appears to use their phone to film or take a picture of the light fixture on the ceiling.
3. A person is in a kitchen. The person is taking a picture of the light in the kitchen.



1. A person puts clothes into their closet on a shelf and then takes out their cell phone.
2. A person takes a pile of clothing into a walk-in close and puts it on a shelf, and they then look at an object they are holding, which might be a cell phone.



1. A person is playing on their phone while sitting on the couch
2. A person is sitting on a couch while looking at a cell phone, and while doing this the person sneezes one time.



1. A person opens a cabinet, takes out a blanket, folds it, opens a closet door, and places the blanket into the closet.
2. A person is walking and then is opening a cabinet. The person takes a towel from the cabinet and folds it. The person opens a closet door and puts the towel away. Lastly, the person walks out of the room.

Abbildung 3.2: Vier Beispielvideos aus dem Charades-Datensatz, mit den zugehörigen Descriptions.

je nach Sampling extrahiert wurden, wird nach dem Max-Similarity-Prinzip vgl. 2.3.2 der Frame für das Ranking ausgewählt. Somit wird sichergestellt, dass jedes Video nur einmal im Ranking erscheint.
Dieser Ablauf wird über alle Messungen beibehalten, um einen möglichst wissenschaftlichen, unabhängigen Vergleich zu ermöglichen.

3.3 Auswahl der Ansätze

3.3.1 Frame-Sampling-Strategien

Im Grundlagenkapitel wurde der Unterschied zwischen festen Abtastraten und inhaltsbasiertem Frame-Sampling dargestellt, vgl. (2.3.1). Da in der Arbeit verschiedene Strategien verglichen werden, mussten für die Strategien verschiedene Ansätze ausgewählt werden. Als Baseline wurde eine feste Abtastrate von einem Frame pro Sekunde gewählt, wie es zum Beispiel Luo et al. vorschlagen [13]. Als Vergleich wurde eine doppelt so hohe Abtastrate von zwei Frames pro Sekunde, welche auch beim VBS Gesamtsiegersystem Vibro [18] von 2023 eingesetzt wurde, und eine halb so hohe von einem Frame alle zwei Sekunden (0,5fps) getestet.

Als Beispiel der inhaltsbasierten Strategie wurde das in Abschnitt 2.3.1 vorgestellte TransNetV2 [23] eingesetzt. Dieses ist ein State of the Art Modell zur Shoterkennung und wurde, nachdem es beim VBS im Jahr 2023 vom System diveXplore [19] eingeführt wurde, in der 2025 Version von fünf Teams eingesetzt [10]. Die Keyframes werden nach den durch TransNetV2 erhaltenen Szenengrenzen ausgewählt, indem der Frame am Mittelpunkt der Szene extrahiert wird. Hat man beispielsweise den Anfang der Szene bei Frame 50 und das Ende bei Frame 168, wäre der Midpoint bei $\frac{50+168}{2} = \underline{109}$.

3.3.2 Modellauswahl

Bei der Architektur des Modells wird auf eine CLIP Architektur gesetzt. Der Grund ist zum einen der gemeinsame Vektorraum in den die Bild- und Text-Vektoren projiziert werden und zum anderen die Möglichkeit die Videos vorzuverarbeiten. Das verschafft vor allem bei Wettbewerben wie dem VBS einen Vorteil. Bei der Modellwahl ist ausschlaggebend, dass eine gute Dokumentation, ein gutes Trainingsframework und offene Pretraining-Checkpoints zur Verfügung stehen. Diese Merkmale vereint die OpenCLIP [11] Modelle. Die Wahl liefert letztendlich das OpenCLIP ViT-B/16 Modell. B steht hierbei für die Base Variante und 16 für die Größe der Patches (2.2.2). Die Base Variante wurde gewählt, da auf der lokalen Hardware lediglich eine Nvidia GTX 1080 mit 8 GB VRAM aus dem Jahre 2016 zur Verfügung steht. Den größeren ViT-L oder ViT-H Modellen bietet die Hardware beim Fine-Tuning, mit brauchbarer Batchsize, nicht ausreichend Platz.
Die Wahl der Patch-Größe fällt auf die 16er-Größe, da die kleineren Patches eine feinere Auflösung als die ebenfalls verfügbare 32er-Größe bieten, aber trotzdem auf der verfügbaren Hardware funktioniert. Als Basismodelle werden zwei Pretrainings-Checkpoints des OpenCLIP ViT-B/16 verwendet. Diese wurden auf dem `datacomp_xl` [8] Pool mit 12,8 Milliarden gesehenen Bild-Text Paaren und dem `datacomp_l`-Pool mit einer 1,28 Milliarden gesehenen Bild-Text Paaren trainiert. Der `datacomp_xl`-Checkpoint gehört zu den stärksten, öffentlichen ViT-B/16 Modellen und erreicht eine ImageNet1K [17] Accuracy von 73,49% [11]. Da beide Checkpoints dieselbe Architektur nutzen und sich nur in der Vortrainingsmenge unterscheiden, erlaubt der Vergleich eine kontrollierte Untersuchung des Einflusses der Vortrainingsbasis.

3.3.3 Fine-Tuning

Nun stellt sich die Frage, warum nicht lediglich das Zero-Shot-Modell verwendet werden sollte, wenn es eine so hohe Accuracy auf wissenschaftlich anerkannten Benchmarks erreicht. Der ImageNet1K ist allerdings ein Bilderbenchmark und sagt nur eingeschränkt etwas über Videodaten aus. Außerdem sind die Captions in Videos ganz anders formuliert als die Captions für Bilder. Die Captions der Videos haben eine zeitliche Komponente, welche das Zero-Shot-Modell noch nicht kennt. Die Herausforderung besteht darin, dass Tests zeigten, dass beim Fine-Tuning von CLIP-Modellen ein sehr hohes Risiko für Overfitting besteht

und damit schnell Robustheit verloren geht [26, 24]. An dieser Stelle kommt die WiSE-FT Methode [27] zum Tragen (2.4.2).

In der vorliegenden Arbeit werden zwei verschiedene Fine-Tuning-Strategien untersucht. Zuerst wird die von Ueki et al. vorgeschlagene Strategie (2.4.4) des partiellen Fine-Tunings (Textencoder vollständig trainieren und Bilderencoder bis auf die letzten zwei Layer einfrieren) verwendet [24]. Außerdem wird zum Vergleich ein komplettes Fine-Tuning durchgeführt.

Die Daten für die Trainingsdurchläufe werden erstellt, indem pro Video ein Frame und eine Caption zufällig, per festem Seed, ausgewählt werden. Genauso machen es Ueki et al. in ihrem Paper [24]. Dies wird gemacht, damit das Training nicht unverhältnismäßig lang auf der lokalen Hardware braucht und damit Redundanz in den Trainingsdaten vermieden wird. Diese entsteht, da unterschiedliche Frames in einem Video oft ähnliche Bilder zeigen.

Bei der Wahl der Hyperparameter wurde zum einen darauf geachtet eine niedrige Lernrate zu verwenden, damit das Pretraining des Modells nicht zerstört wird und zum anderen wird die Batch Size auf die Größe 32 gesetzt, die durch die 8GB VRAM limitiert wird. Außerdem wird die Anzahl der Epochen so gesetzt, dass ein klarer Overfitting-Verlauf beobachtet werden kann.

Eine weitere wichtige Entscheidung trifft die Wahl des Trainingscheckpoints für den weiteren Verlauf der Arbeit. Während des Trainingslaufs wird nach jeder Epoche mit dem Evaluation-Set vom MSR-VTT verglichen. Denn obwohl der Trainingsloss mit jeder Epoche weiter sinkt, wird der Validationsloss ab einem gewissen Punkt steigen. An diesem Punkt hat das Modell den Trainingsdatensatz auswendig gelernt und es kommt zum Overfitting. Jedoch wird nicht nur der Validationsloss untersucht, um die richtige Epoche zu finden, sondern auch die anderen Metriken wie Recall@1,5 und 10; Median-Rank und der Mean-Rank. Damit wird unter den trainierten Epochen die gewählt, die im Zusammenspiel das vielversprechendste Ergebnis erzielt.

3.3.4 Gewichtsinterpolation

Dank des Fine-Tunings sind nun beide Checkpoints vorhanden, was als Voraussetzung dient um die Gewichtsinterpolation durchzuführen. Damit der Parameter α (2.3.3) genauer untersucht werden kann, wird ein systematischer Sweep über alle $\alpha \in \{0.0, 0.1, 0.2, ..., 1.0\}$ durchgeführt. Damit wird empirisch geprüft, welcher α-Wert die beste Retrieval-Qualität liefert. Folglich werden pro Modell, über alle elf α-Werte und Frame-Strategien, der gesamte Test-Datensatz des MSR-VTT durchlaufen und alle Metriken berechnet. Das Entspricht:

$$2 \text{ Fine-Tuning-Strategien} \times 4 \text{ Frame-Strategien} \times 11 \ \alpha\text{-Werte} = 88$$

Testdurchläufen pro Basismodell, also insgesamt 176 Durchläufen, mit knapp 3000 Testvideos.

3.3.5 Vektorsuche

Da bei der Vektorsuche auch der am besten zum Text passende Vektor gesucht wird, findet auch hier das unter Punkt 2.3.2 erklärte Max-Similarity-Prinzip Anwendung. Für die Suche wird auf die FAISS-Bibliothek [7] zurückgegriffen.

Als Ähnlichkeitsmaß dient die in Abschnitt 2.1.2 beschriebene Kosinus-Ähnlichkeit. Da FAISS allerdings keinen dedizierten Index für die Kosinus-Ähnlichkeit zur Verfügung stellt, wird ausgenutzt, dass das Skalarprodukt zweier normalisierter Vektoren der Kosinus-Ähnlichkeit entspricht. Hierfür werden die Feature-Vektoren vor dem Einsatz von FAISS normiert und über einen Skalarprodukt-Index verglichen. Dieser Zusammenhang lässt sich mathematisch wie folgt darstellen:

$$\text{Kosinus-Ähnlichkeit}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|} = \left(\frac{\vec{a}}{\|\vec{a}\|} \right) \cdot \left(\frac{\vec{b}}{\|\vec{b}\|} \right) = \vec{a}_{\text{norm}} \cdot \vec{b}_{\text{norm}}$$

wobei $\vec{a}_{\text{norm}}$ und $\vec{b}_{\text{norm}}$ die auf die Länge 1 skalierten Vektoren (Einheitsvektoren) repräsentieren.

Da der Test-Datensatz noch eine überschaubare Größe mit ca. 3000 Videos besitzt, wird eine exakte statt einer approximativen Suche gewählt. Dies vermeidet Approximationsfehler, die andernfalls den Vergleich der verschiedenen Konfigurationen stören könnten. Außerdem stellt dies sicher, dass die ermittelten Leistungsunterschiede allein auf die untersuchten Faktoren zurückzuführen sind.

3.4 Systementwurf

3.4.1 Technologien

Bei der Umsetzung aller Anforderungen in Form des Prototypen wurden folgende Technologien und Frameworks verwendet. Eine Kurzfassung dieser liegt bei Tabelle 3.1 vor.

Als Programmiersprache wurde Python 3.14 verwendet. Die Programmiersprache ist der Standard im Machine-Learning-Bereich und beinhaltet alle notwendigen Frameworks. Außerdem sind die notwendigen Technologien wie OpenCLIP, TransNetV2 und die FAISS Bibliothek hauptsächlich für Python verfügbar, was diese de facto zum Standard im ML-Bereich macht. Als Deep Learning Framework wird PyTorch verwendet, welches auch das zugrundeliegende Framework von OpenCLIP ist. Außerdem beinhaltet es CUDA-Unterstützung, die benötigt wird, um die GPU zu nutzen. OpenCLIP, wie schon in Abschnitt 3.3.2 dargestellt, ermöglicht das Laden, Fine-Tuning und die Interpolation des Modells. Für die Extraktion der Frames bei den festen Abtastraten wird die Bibliothek OpenCV verwendet. OpenCV ist eine sehr umfangreiche Bibliothek für Bildverarbeitung, Computergrafik und maschinelles Lernen und wird auch von teilnehmenden Systemen wie

VERGE [15] beim VBS eingesetzt. Zur Extraktion der inhaltsbasierten Keyframes wird, wie in Abschnitt 3.3.1 erläutert, TransNetV2 eingesetzt. Die in Abschnitt 3.3.5 eingeführte FAISS Bibliothek, stellt den Vektorindex der Embedding-Vektoren und die Ähnlichkeitssuche für den Prototypen bereit. NumPy beinhaltet eine hohe Anzahl an mathematischen Funktionen und wird in dem Prototypen für die Vektoroperationen und die Speicherung der Embeddings verwendet. Zur Erstellung der Grafiken, zur Veranschaulichung der Ergebnisse, wird die Bibliothek matplotlib verwendet. Für die Benutzeroberfläche wird als Bibliothek Streamlit verwendet. Diese ermöglicht die sehr simple Erstellung einer Web-App in Python ohne separates Front-End und ist ohne großzügige Einarbeitung einfach zu verstehen. Gewählt wurde diese Bibliothek, da die gesamte Pipeline bereits in Python bereitsteht und sich Streamlit somit nahtlos einbinden lässt. Zudem werden alle benötigten Elemente als fertige Komponenten bereitgestellt. Für unterstützende Aufgaben kamen außerdem die Bibliotheken rich für Fortschrittsanzeigen, sowie Pillow für das Laden der Bilddateien in der Benutzeroberfläche zum Einsatz.

Tabelle 3.1: Technologien und Frameworks des Prototypen

Komponente	**Technologie**	**Begründung**
Programmiersprache	Python 3.14	Standard im ML-Umfeld, großes Ökosystem
DL-Framework	PyTorch	Basis von OpenCLIP, CUDA-Unterstützung
Modell-Framework	OpenCLIP	Modellverarbeitung (3.3.2)
Frame-Extraktion	OpenCV	Auslesen fester Abtastraten
Szenenerkennung	TransNetV2	inhaltsbasiertes Frame-Sampling (3.3.1)
Vektorsuche	FAISS	Vektorindexierung und Ähnlichkeitssuche (3.3.5)
Numerische Operationen	NumPy	Vektoroperationen
Visualisierung	matplotlib	Diagramme der Evaluationsergebnisse
Benutzeroberfläche	Streamlit	interaktive Such-App in reinem Python

3.4.2 Architektur

Die Architektur des Prototypen ist grundsätzlich modular aufgebaut, sodass jede Stufe einzeln konfigurier- und ausführbar ist. Dabei ist sie in 2 Phasen aufgeteilt:

1. Die Vorverarbeitung
2. Die Suche

Bei der Vorverarbeitung werden die Videos vom MSR-VTT-Datensatz verarbeitet und mit den verschiedenen Frame-Sampling Strategien in Keyframes zerlegt. Daraufhin werden

diese Keyframes durch die verschiedenen interpolierten CLIP-Modelle geschickt, um die Embedding Vektoren zu erstellen. Diese werden dann von FAISS in einem Vektorindex gespeichert.

Daran anschließend erfolgt die Suche, bei der zuerst die Textanfrage in der Benutzeroberfläche entgegengenommen und ebenfalls durch das interpolierte CLIP-Modell (den Text Encoder) geschickt wird. Anschließend wird mit dem berechneten Vektor die Ähnlichkeitssuche ausgeführt und die ähnlichsten Keyframes aus jedem Video gesucht (Max-Similarity). Daraufhin werden die besten K Ergebnisse in der App angezeigt.

Abbildung 3.3 zeigt die Architektur als Abbildung.

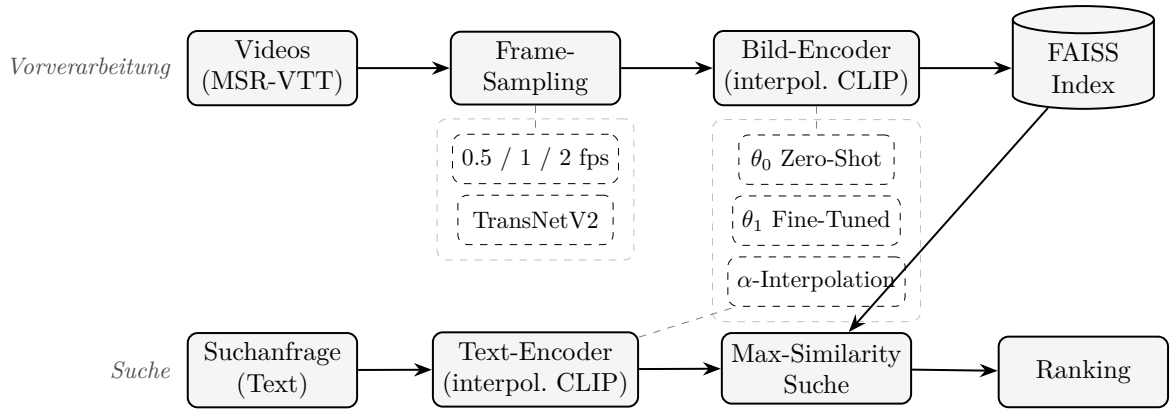


Abbildung 3.3: Architektur des Python-Prototypen. Die gestrichelt umrandeten Komponenten bilden die in dieser Arbeit untersuchten Optimierungsachsen.

Zentrale Konfiguration

Damit die Konsistenz unter den einzelnen Stufen haltbar ist, gibt es eine zentrale Stelle welche die wichtigsten Einstellungen für das Modell, die Frame-Strategie und andere Faktoren beinhaltet. Damit wird garantiert, dass alle Module dieselbe Konfiguration nutzen und verhindert Abweichungen der Experimente und damit auch der Ergebnisse. Das sichert die Reproduzierbarkeit der Ergebnisse, damit andere Benutzer sich nicht durch das ganze System arbeiten müssen.

Modularität und Erweiterbarkeit

Damit die Anforderung der Erweiterbarkeit erfüllt werden kann, muss das System so aufgebaut sein, dass die einzelnen Stufen leicht angepasst und ausgetauscht werden können, damit wird die Integration von zum Beispiel einem neuen Datensatz oder einer anderen Sampling-Strategie erleichtert. Dazu ist das System in verschiedene Module aufgeteilt, die jeweils eine Stufe der Pipeline abbilden. Diese Module sind über klar definierte Schnittstellen

miteinander verbunden, sodass Änderungen an einem Modul keine Auswirkungen auf die anderen haben.

Interaktive Anwendung

Um die Funktionalität des Prototypen zu demonstrieren, dient die Streamlit-App. Diese ermöglicht eine Textsuche mit Auswahl von Frame-Strategie und Modell, sowie einen Vergleichsmodus, um zwei Konfigurationen zu vergleichen. Die Details zur Implementierung folgen in Kapitel 4.

Kapitel 4: Implementierung

4.1 Übersicht

Im folgenden Kapitel wird die konzeptionelle Pipeline aus Kapitel 3 auf die tatsächlich implementierten Module übertragen. Tabelle 4.1 zeigt eine Übersicht aller Module des Prototypen.

Tabelle 4.1: Module des Prototypen mit ihren Aufgaben

Modul	**Aufgabe**
`config.py`	Hält die Einstellungen zentral, leitet daraus die Ausgabepfade ab und stellt Ladefunktion der Modelle bereit.
`create_frames_time.py`	Extrahiert Keyframes bei den festen Abtastraten.
`create_frames_transnetv2.py`	Ermittelt die Szenengrenzen mit TransNetV2 und extrahiert den Frame am Szenenmittelpunkt.
`implement_wiseft.py`	Erzeugt die interpolierten Modelle für alle α-Werte aus dem Zero-Shot- und dem feingetunten Checkpoint.
`build_image_vectors.py`	Encodiert die Keyframes und speichert die normalisierten Embeddings mit ihren Metadaten.
`build_faiss_index.py`	Baut aus den Embeddings den Vektorindex auf.
`evaluate_models.py`	Bestimmt für jede Anfrage den Rang des korrekten Videos und berechnet daraus die Metriken.
`search.py`	Implementiert eine simple Suche per CLI Input zum Testen.
`wise_sweep.py`	Erledigt automatisiert die Evaluation über alle Konfigurationen und ruft dazu die vorherigen Module auf.
`app.py`	Stellt die interaktive Suche mit Einzel- und Vergleichsmodus bereit.

Wichtig zu Beginn ist die Bedeutung der zentralen Konfiguration des Systems. In der Datei `config.py` werden alle wichtigen Einstellungen des Prototypen vorgenommen, welche die anderen Module importieren. Hier werden die Einstellungen für den Modellnamen des Zero-Shot-Modells, die verwendeten Pretraining-Checkpoints, die Modellquelle, alle wichtigen

Verzeichnisse für andere Module, der verwendete Modell-Tag zur genauen Identifikation des Modells und weitere Einstellungen vorgenommen. Aus dem Modell-Tag werden die Pfade zum Ordner des jeweiligen Modells erstellt, worin die Embeddings der Bilder, deren Index und die jeweiligen Metadaten zur Identifikation des korrekten Frames für die spätere Evaluation liegen. Außerdem bietet `config.py` die Möglichkeit, während der Laufzeit diese Pfade anzupassen, um den späteren Sweep über alle α-Werte durchzuführen, ohne jeweils händisch eingreifen zu müssen. Ebenfalls bietet die Config eine einheitliche Lade-Funktion für das CLIP-Modell.

4.2 Vorverarbeitung

Das Ziel der Vorverarbeitung ist, aus den Videodaten Keyframes zu erzeugen, die im späteren Verlauf eindeutig einem Video zuzuordnen sind.

4.2.1 Feste Abtastraten

Für die Extraktion der festen Abtastraten ist das Modul `create_frames_time.py` zuständig. Dieses iteriert durch den Ordner mit den Videos und ruft für jedes Video eine Funktion auf, welche die Keyframes extrahiert. Dazu wird das Video mit Hilfe von OpenCV geöffnet. Da die Frameraten der Videos variieren, muss pro Video das Intervall zum Extrahieren der Frames neu berechnet werden. Dazu wird die Bildrate des Videos erfragt und durch die gewollte Abtastrate geteilt. Anschließend wird über die einzelnen Bilder im Video über das berechnete Intervall iteriert und die Keyframes werden als JPEG gespeichert. Hervorzuheben ist das Namensschema, welches aus Video-ID und Bildindex besteht und die spätere Zuordnung der Bilder garantiert. Alle drei Bildraten (2fps, 1fps, 0,5fps) werden vom gleichen Modul mit unterschiedlichem Parameter erstellt und müssen so drei mal aufgerufen werden.

Da das Frame-Interval eine ganzzahlige Nummer als Ergebnis benötigt, wird das Ergebnis der Division von Video-Bildrate durch die gewollte Abtastrate auf Integer gecastet. Das ist notwendig da einige Videos keine ganzzahligen Bildraten haben. Das hat zur Folge, dass zwei Bilder pro Sekunde nicht unbedingt die doppelte Frameanzahl wie das 1fps-Sampling liefern, ebenso verhält es sich bei 0,5fps.

4.2.2 TransnetV2

Bei der inhaltsbasierten Keyframe Extraktion wurde, wie in Abschnitt 3.3.1 erläutert, auf das TransNetV2 Modell gesetzt. Zuerst mussten, um das Modell mit Pytorch benutzen zu können, die Tensorflow Gewichte in Pytorch Gewichte umgewandelt werden. Im offiziellen GitHub Repository des TransNetV2 Papers [22] findet sich eine Anleitung, die sowohl die Umwandlung der Gewichte, als auch den Code für die Benutzung des Modells abdeckt. Da

TransNetV2 die Videos nur in einem bestimmten Format annimmt, mussten die Videos vor der Nutzung konvertiert werden. Die vom Modell gelieferten Wahrscheinlichkeiten für einen Szenenübergang wurden mithilfe eines Threshold von 0,5 und einer minimalen Szenenlänge von fünf Frames in einzelne Szenen unterteilt. Von den Szenen wurde der Mittelpunkt extrahiert und als Bild mit dem gleichen Namensschema wie bei den festen Abtastraten gespeichert. Diese Namensgebung ermöglicht einen gleichen Umgang mit allen Abtastraten im späteren Ablauf der Evaluation.
Die Umsetzung dieses Moduls befindet sich in `create_frames_transnetv2.py`.

4.2.3 Datenvolumen

Für eine spätere Effizienzbewertung der Frame-Sampling-Strategien muss das entstehende Datenvolumen untersucht werden. Nach der Extraktion der Keyframes aus dem MSR-VTT-Datensatz ergeben sich die in Tabelle 4.2 dargestellten Zahlen. Dabei ist darauf hinzuweisen, dass beispielsweise 2fps nicht genau das doppelte Datenvolumen vom 1fps-Sampling liefert, was dem Integer-Casting zuzuschreiben ist (vgl. 4.2.1).

Tabelle 4.2: Speicherbedarf der Frame-Sampling-Strategien für den gesamten MSR-VTT-Datensatz

Strategie	**Speicherbedarf**	**Dateianzahl**
1fps	3,94 GB	154.358
2fps	8,08 GB	316.054
0,5fps	2,02 GB	79.588
TransNetV2	853 MB	34.023

4.2.4 Charades

Für den OOD-Test mussten die Keyframes für den Datensatz Charades extrahiert werden. Die Frames wurden mit dem bestehenden Modul extrahiert, allerdings waren die gegebenen Captions in einem anderen Format als beim MSR-VTT-Datensatz. Außerdem bietet der Charades-Datensatz zwei Arten von Captions, zum einen das Script und zum anderen die Descriptions. Um den Charakter des OOD-Tests zu wahren, wurden die Descriptions gewählt, da diese von Menschen erstellt und deutlich länger als die Captions des MSR-VTT-Datensatzes sind. Außerdem dient das Script als eine Anleitung an den Ersteller des Videos, während die Descriptions das Video beschreiben. Aus diesem Grund passt es besser zum Setting des VBS, bei dem das System das Video ebenfalls anhand einer Beschreibung finden muss. Somit wurden aus der gegebenen CSV die VideoID's und die Descriptions extrahiert und in eine JSON geschrieben. Dadurch wird die Verwendung der vorhandenen Pipeline auch für den OOD-Test ermöglicht.

4.3 Modellerzeugung

4.3.1 Aufbereitung der Trainingsdaten

Zum Trainieren des OpenCLIP Vit-B/16 Modells wurde das Trainingsmodul von OpenCLIP genutzt. Um die Verwendung zu ermöglichen, mussten zwei CSV Dateien im richtigen Format erstellt werden. In der CSV musste eine Spalte den Bildpfad und eine Spalte die Caption enthalten. Hierfür wurde ein zufälliger Frame aus dem 1fps extrahierten Frames und eine zufällige Caption pro Video ausgewählt und in die CSV geschrieben. Das geschah über einen festen Seed, um die Reproduzierbarkeit zu wahren. Wichtig hervorzuheben ist, dass das Fine-Tuning nichts mit den Frame-Strategien zu tun hat, die nur Index und Evaluation betreffen. Dabei ergaben sich 6513 Trainings- und 497 Evaluationspaare.

4.3.2 Durchführung des Fine-Tunings

Wie bereits aufgezeigt, wurde das auf dem offiziellen GitHub Repository dokumentierte OpenCLIP Trainingsmodul [12] benutzt. Folglich existiert kein eigener Trainingscode. Zum Training wurden folgende Hyperparameter angegeben:

- Eine niedrige Lernrate von 1e-5, um die Pretraining-Gewichte nicht zu zerstören
- Eine Batch-Größe von 32, angepasst an den kleinen Grafikspeicher
- 10 Epochen, um den Overfitting-Verlauf zu untersuchen und eine richtige Epoche im Nachhinein auszuwählen
- Eine Speicherfrequenz von jeder Epoche, um jede Epoche als Auswahl zu haben
- Eine Validierungsfrequenz von jeder Epoche
- AdamW als Optimierungsalgorithmus

Die übrigen Parameter wurden auf den Standardwerten belassen. Zusätzlich mussten Name, Speicherort, Modellarchitektur, Pretraining-Checkpoint und die Dateipfade und Informationen zu den Trainings- und Validationsdateien angegeben werden.

Mit diesem Setup wurde das volle Fine-Tuning ausgeführt. Beim partiellen Fine-Tuning wurde zusätzlich der Image-Encoder eingefroren und direkt 2 Blöcke wieder freigegeben. Damit wurde erreicht, dass der gesamte Text-Encoder und nur die letzten beiden Layer des Image-Encoders trainiert wurden. Somit unterscheiden sich beide Trainingsstrategien nur in diesen beiden unterschiedlichen Flags. Aufgrund dieser Tatsache stellt sich der Vergleich beider Strategien als so aussagekräftig dar.

Beim Schreiben der Arbeit zeigte sich, dass mit den Standardwerten das Warmup auf 10.000 Schritte gesetzt ist. Dies hat zur Auswirkung, dass bei der verwendeten Datenmenge, nur der Aufwärmbereich durchlaufen wurde, was das Erreichen der angegebenen Lernrate

verhindert. Folglich wurde, bis dahin unbemerkt, nur mit einer Lernrate von bis zu 2e-6 trainiert, da diese im Warmup linear von null auf steigt. Für die interne Vergleichbarkeit der Modelle war das nicht relevant, da das Training aller Modelle unter den gleichen Bedingungen verlief. Auch das Fine-Tuning zeigte sich gegenüber der niedrigen Lernrate als wirksam, siehe Abschnitt 4.3.3.

4.3.3 Auswahl der Epochen

Die Auswahl der Checkpoints wurde nach dem im Abschnitt 3.3.3 erklärten Prinzip getroffen. Die vier Trainingsverläufe sind in den Tabellen 4.3 und 4.4 aufgeführt.

Tabelle 4.3: Fine-Tuning Ergebnisse der zwei Strategien mit dem datacomp_l Checkpoint

Partial Fine-Tuning; `datacomp_l`

			Image-to-Text				Text-to-Image			
Ep.	**Tr.L.**	**Val L.**	**Mn**	**Md**	**R@1**	**R@5/10**	**Mn**	**Md**	**R@1**	**R@5/10**
1	1.7170	2.2020	39.5	7	0.2193	0.4708 / 0.5674	45.2	6	0.2414	0.4889 / 0.5674
2	1.7628	1.9655	37.1	6	0.2374	0.4930 / 0.6016	41.9	6	0.2636	0.4950 / 0.5775
3	1.5244	1.7335	35.2	5	0.2535	0.5010 / 0.6177	38.1	**5**	0.2716	0.5030 / 0.5875
4	1.0531	1.5393	33.8	5	0.2736	0.5332 / 0.6217	34.6	**5**	0.2757	0.5191 / 0.6056
5	1.0379	1.4084	32.3	**4**	0.2636	0.5312 / 0.6398	32.4	**5**	0.2777	0.5292 / 0.6076
6	1.4334	1.3356	30.9	**4**	0.2636	0.5473 / 0.6358	30.8	**5**	0.2777	0.5292 / 0.6278
7	0.9383	1.2929	30.0	**4**	0.2676	0.5533 / 0.6419	29.1	**5**	0.2817	0.5312 / 0.6298
8	1.0521	1.2712	29.4	**4**	**0.2757**	0.5533 / 0.6459	28.2	**5**	0.2817	0.5231 / 0.6419
9	1.0403	**1.2616**	29.0	**4**	0.2636	0.5453 / 0.6479	27.5	**5**	0.2817	0.5332 / **0.6479**
10	**0.7383**	1.2685	**28.6**	**4**	0.2696	**0.5594** / **0.6579**	**27.2**	**5**	**0.2837**	**0.5352** / 0.6398

Full Fine-Tuning; `datacomp_l`

			Image-to-Text				Text-to-Image			
Ep.	**Tr.L.**	**Val L.**	**Mn**	**Md**	**R@1**	**R@5/10**	**Mn**	**Md**	**R@1**	**R@5/10**
1	1.7079	2.1639	39.0	7	0.2213	0.4728 / 0.5795	44.4	6	0.2394	0.4829 / 0.5674
2	1.6826	1.8803	36.2	6	0.2435	0.4970 / 0.6036	39.8	6	0.2656	0.4970 / 0.5835
3	1.4397	1.6387	34.3	5	0.2596	0.5191 / 0.6177	35.3	5	0.2777	0.5030 / 0.5915
4	0.9492	1.4599	32.9	**4**	0.2696	0.5392 / 0.6237	32.2	5	0.2837	0.5252 / 0.6157
5	0.9346	1.3523	31.5	**4**	0.2676	0.5453 / 0.6398	30.1	5	0.2837	0.5171 / 0.6258
6	1.2257	1.2965	30.5	**4**	0.2696	0.5493 / 0.6559	28.9	5	0.2797	0.5312 / 0.6479
7	0.7601	**1.2712**	29.4	**4**	0.2716	**0.5654** / 0.6579	27.8	5	0.2897	0.5231 / 0.6559
8	0.8698	1.2799	29.0	**4**	**0.2736**	0.5594 / 0.6600	**27.4**	5	**0.2918**	0.5332 / 0.6439
9	0.7289	1.2955	28.9	**4**	0.2696	0.5513 / **0.6620**	27.5	**4**	0.2857	**0.5392** / 0.6559
10	**0.3961**	1.3556	**28.8**	**4**	**0.2736**	0.5392 / 0.6600	27.7	5	0.2857	0.5272 / **0.6579**

Bei Betrachtung der Ergebnisse ist auffallend, dass die Metriken ihr Optimum nicht an derselben Epoche erreichen. Dies unterstreicht die Wahl der Epoche anhand verschiedener Metriken. Eine weitere Auffälligkeit ist, dass der Trainingsloss bei allen Modellen auffallend stark von der neunten zur zehnten Epoche sinkt, obwohl der Validationloss ansteigt. Ein klares Indiz dafür, dass es zu Overfitting kommt. Die Wahl der Epochen fällt bei dem vollen Fine-Tuning und dem `datacomp_l` Basismodell auf Epoche 7, da dies der Tiefpunkt des

Tabelle 4.4: Fine-Tuning Ergebnisse der zwei Strategien mit dem datacomp_xl Checkpoint

Partial Fine-Tuning; `datacomp_xl`

			Image-to-Text				Text-to-Image			
Ep.	**Tr.L.**	**Val L.**	**Mn**	**Md**	**R@1**	**R@5/10**	**Mn**	**Md**	**R@1**	**R@5/10**
1	1.6190	1.8391	33.0	4	0.2938	0.5513 / 0.6318	38.4	4	0.3139	0.5453 / 0.6378
2	1.4109	1.6060	30.2	4	0.3078	0.5815 / 0.6539	35.6	4	0.3300	0.5654 / 0.6539
3	1.2376	1.3978	28.1	**3**	0.3280	0.5915 / 0.6720	31.9	**3**	**0.3340**	0.5835 / 0.6660
4	0.8680	1.2406	26.5	**3**	0.3239	0.5936 / 0.6901	29.1	**3**	0.3300	0.5976 / 0.6700
5	0.8635	1.1487	25.0	**3**	0.3320	0.6036 / 0.7002	26.9	**3**	0.3260	0.5956 / 0.6841
6	1.0953	1.1115	24.0	**3**	0.3320	0.6056 / 0.7082	25.7	**3**	0.3199	0.6016 / 0.6881
7	0.7627	**1.0974**	**23.7**	**3**	0.3340	**0.6137** / 0.7062	24.8	**3**	0.3260	0.6056 / **0.6901**
8	0.7889	1.1035	23.8	**3**	**0.3360**	0.6036 / **0.7103**	24.6	**3**	0.3239	0.6076 / 0.6861
9	0.7743	1.1198	24.0	**3**	**0.3360**	0.6036 / 0.6982	24.6	**3**	0.3219	**0.6197** / **0.6901**
10	**0.4486**	1.1480	24.5	**3**	0.3280	0.5976 / 0.6861	**24.4**	**3**	0.3260	0.6036 / 0.6801

Full Fine-Tuning; `datacomp_xl`

			Image-to-Text				Text-to-Image			
Ep.	**Tr.L.**	**Val L.**	**Mn**	**Md**	**R@1**	**R@5/10**	**Mn**	**Md**	**R@1**	**R@5/10**
1	1.6019	1.7991	32.5	4	0.2918	0.5513 / 0.6419	37.5	4	0.3219	0.5493 / 0.6459
2	1.3477	1.5171	29.1	4	0.3219	0.5875 / 0.6640	33.3	**3**	0.3421	0.5855 / 0.6559
3	1.1355	1.3038	26.9	**3**	0.3239	0.6016 / 0.6841	29.1	**3**	0.3380	0.5936 / 0.6660
4	0.7411	1.1722	25.4	**3**	0.3340	0.6056 / 0.6901	26.8	**3**	0.3421	0.5895 / 0.6781
5	0.7457	1.1064	23.8	**3**	0.3260	0.6076 / **0.7163**	24.7	**3**	0.3400	0.5996 / 0.6781
6	0.8959	**1.0982**	23.3	**3**	0.3199	0.6117 / **0.7163**	24.3	**3**	**0.3461**	0.6117 / 0.6801
7	0.5509	1.1158	**23.0**	**3**	0.3300	**0.6177** / 0.7123	**24.3**	**3**	0.3400	**0.6157** / **0.6861**
8	0.5863	1.1554	23.4	**3**	**0.3360**	0.6056 / 0.7103	24.4	**3**	0.3400	0.5895 / 0.6821
9	0.4046	1.2258	24.1	**3**	0.3260	0.5875 / 0.6962	24.8	**3**	0.3380	0.6117 / **0.6861**
10	**0.1639**	1.3034	24.7	**3**	**0.3360**	0.5835 / 0.6942	25.6	**3**	0.3400	0.5936 / 0.6761

Validationloss ist. Der Trainingsloss sinkt im Folgenden noch weiter. Das ist ein eindeutiger Anhaltspunkt dafür, dass das Modell die Trainingsdaten auswendig gelernt hat. Das beste Recall@1 Ergebnis liegt eine Epoche weiter mit einem winzigen Abstand. Dieser ist bei der kleinen Anzahl von Validierungsvideos zu vernachlässigen.
Beim partiellen Fine-Tuning und gleichem `datacomp_l` Basismodell war der Verlauf sehr flach, sodass sich kein ausgeprägtes Loss-Minimum abbildet und der Wert fortlaufend kleinschrittig sank. Im Nachhinein ließ sich das vor allem auf den größtenteils eingefrorenen Image-Encoder und die durch das Warmup geprägte, niedrige Lernrate zurückführen. Auch die Recall@K Werte lagen über die letzten Epochen sehr nah beieinander. Nach diesen Beobachtungen wurde entschieden, dieselbe Epoche wie beim vollen Fine-Tuining zu verwenden, um eine noch bessere Vergleichbarkeit zu erlangen.

Bei den `datacomp_xl` Modellen war die Entscheidung eindeutiger. Die Ergebnisse des vollen Fine-Tunings zeigten beim Validation Loss ein Minimum bei Epoche 6. Gleichzeitig ist der Recall@1 Wert in der Text-zu-Bild-Richtung dort am höchsten. Interessant ist an diesem Punkt zu beobachten, dass das Optimum eine Epoche früher auftitt, als bei dem schlechter vortrainiertem Modell `datacomp_l`.

Beim partiellen Fine-Tuning lag der höchste Recall@1 Wert für die voll trainierte Text-zu-Bild-Richtung frühzeitig bei Epoche 3. Im Gegensatz dazu lagen die Optima der anderen Metriken wie dem Validation Loss jedoch bei Epoche 7 und später. Die Mehrheit der Indikatoren weisen demnach auf eine spätere Epoche. Im Folgenden wird für das partielle Fine-Tuning beim `datacomp_xl` Modell Epoche 7 gewählt.

4.3.4 Gewichtsinterpolation

Die Gewichtsinterpolation wurde mit dem Modul `implement_wiseft.py` umgesetzt. Dazu wurden das Zero-Shot- und das feingetunte Modell geladen und geprüft, ob beide Modelle kompatibel sind. Daraufhin wurden alle Fließkommazahlen des Modells via der WiSE-FT Formel interpoliert(vgl. 2.3.3). Erwähnenswert ist, dass OpenCLIP-Modelle auch nicht-gleitkommawertige Einträge besitzen, die sich nicht mischen lassen. Diese wurden vom feingetuntem Modell übernommen(vgl. 4.1).

```
if t0.is_floating_point():
  theta_interp[key] = (1 - alpha) * t0 + alpha * t1
else:
  theta_interp[key] = t1
```

Codeauszug 4.1: Implementierung der WiSE-FT-Interpolation

Die interpolierten Modelle wurden folgend im selben Format wie die originalen Modelle gespeichert, damit die Ladefunktion nicht zwischen interpolierten und originalen Modellen unterscheiden muss. Dieser Ablauf wurde für alle feingetunten Modelle für jedes α durchgeführt.

4.4 Indexierung und Suche

4.4.1 Bild-Embeddings und Index

Zum Erzeugen der Bild-Embeddings wurde das Modul `build_image_vectors.py` verwendet. Dazu wurden zuerst die IDs der Test Videos aus der Test-Liste entnommen und damit jeweils alle zum Video gehörenden Frames aufgefunden. Dank der Vorverarbeitung lief dieses Schema bei jeder Strategie einheitlich und erforderte keine händischen Eingriffe. Daraufhin wurden die Bilder batchweise durch das Modell encodiert. Die entstandenen Feature-Vektoren wurden normiert und in einer Datei gespeichert. Außerdem wurde eine Metadaten-Datei gespeichert, die jeweils die VideoID und den Dateipfad zum Keyframe für jeden durchlaufenen Frame enthielt. Das ermöglichte erst die genaue Zuordnung von Frame zu Video sowie die spätere Trefferanzeige in der App.
Folgend wurde der Index mit FAISS, als exakte Suche über die Skalarprodukte mit 512

Dimensionen gebaut. Um die Speicherqualität bewerten zu können, soll nicht nur der Speicheraufwand der tatsächlichen Frames untersucht werden, sondern auch die der Indizes. Die Tabelle 4.5 gibt Auskunft über die Größe der Indizes für den MSR-VTT Test-Split.

Tabelle 4.5: Speicheraufwand der Indizes beim MSR-VTT-Test-Split mit 2990 Videos

Strategie	**Frames im Index**	**Indexgröße**
1fps	46.708	91,2 MiB
2fps	95.729	187 MiB
0,5fps	24.074	47,0 MiB
TransNetV2	10.064	19,7 MiB

Dieser Vorgang wurde für jede Kombination aus Modell und Strategie wiederholt. Wie bereits in 3.3.4 errechnet ergab das 88 Indizes pro Basismodell, also 176 Indizes für den MSR-VTT-Datensatz, plus 22 für den OOD Test mit Charades.

4.4.2 Anfrageverarbeitung

Der Anfragetext wird durch das gleiche Modell wie die Bilder encodiert und normalisiert. Im Folgenden wird die Ähnlichkeitssuche im Index ausgeführt. Bei der einfachen Suche, wie in der App, werden die Indizes berechnet und pro Video immer der ähnlichste Frame gespeichert (Max-Similarity 2.3.2). Dazu wird ein Set für die gesehenen Videos erstellt und über alle Ausgaben der Ähnlichkeitssuche iteriert. Erscheint ein Frame, dessen zugehöriges Video noch nicht in den gesehenen Videos ist, wird das Video dem Set hinzugefügt und ein neuer Eintrag in dem Ergebnis Array erstellt. Die Suche wird fortgeführt, bis eine gewünschte Anzahl von Ergebnissen gefunden wurde, also das Set die gewünschte Länge hat.

4.5 Evaluation und Automatisierung

4.5.1 Evaluationsablauf

Die Umsetzung des Evaluationsablaufs beginnt mit der Vorbereitung der Textanfragen. Für jedes Video wird eine der circa 20 Captions pro Video beim MSR-VTT zufällig, per festem Seed für die Reproduzierbarkeit, extrahiert. Dabei werden nur Videos genutzt, die in der Test-Split-Liste enthalten sind, da nur diese auch im vorbereitetem Index vorhanden sind. Im nächsten Schritt werden die Anfragetexte sowie die Keyframes batchweise encodiert und die Merkmalsvektoren normiert. Mit diesen erfolgt pro Textanfrage die Ähnlichkeitssuche

sowie die schon im vorherigen Abschnitt erläuterte Rangbestimmung pro Video. Im Unterschied zur vorher beschriebenen Anfrageverarbeitung wird bei dem Evaluationsablauf nicht nur eine bestimmte Anzahl an Indizes extrahiert, sondern es wird der gesamte Index durchsucht. Dadurch wird sowohl die Berechnung der Top-K Ränge, als auch die Berechnung der anderen Metriken (Median- und Mean-Rank) ermöglicht. Aus diesen Rängen werden die, in Abschnitt 3.2.2 erläuterten, Metriken berechnet.

Abschließend werden die berechneten Metriken in eine CSV-Datei mit allen Ergebnissen angehängt. Das Modell-Tag wird als Kennung verwendet. Dadurch wird die Automatisierung des Vorganges erleichtert und Fehler beim Abtippen oder Herauskopieren vermieden.

4.5.2 Automatisierung

Aufgrund der hohen Anzahl an Messungen bei zwei Basismodellen mit jeweils zwei Fine-Tuning-Strategien, vier Frame-Sampling-Strategien und elf α-Werten, wäre eine manuelle Ansteuerung jedes Laufes sowohl aufwendig, als auch sehr fehleranfällig. Aus diesem Grund wurde eine einfache Automatisierung entworfen, welche die Iteration über die α-Werte und die Frame-Strategien implementiert. Dazu erfolgte die Anpassung von `config.py` während der Laufzeit, wobei eine simple Methode verwendet wurde. Das Modul zur Automatisierung importiert hierzu die bereits fertigen Module zur Embedding-Erzeugung (`build_image_vectors`), den Indexaufbau (`build_faiss_index`) und Evaluation (`evaluate_models`) und ruft deren Hauptfunktion auf. Das veranschaulicht die funktionierende Modularität des Prototypen und erfüllt damit die nicht funktionale Anforderung.

Es wurde beachtet, dass im Falle eines Abbruchs oder Absturzes des Programmes, dieses seine Arbeit beim nächsten Aufruf fortsetzen kann. Dafür wird zuerst die Existenz des Index überprüft und im Anschluss unter der Verwendung des Modell-Tags, ob zu dem zurzeit evaluiertem Modell schon ein Eintrag in der Ergebnistabelle existiert.

Zur Einordnung und Veranschaulichung des Aufwands der Durchführung:
Die reine Laufzeit eines Durchlaufs eines feingetunten Modells ohne Eingriffe, Abbrüche und Tests läuft circa 15,4 Stunden. Die Aufteilung erfolgt in $11 \times (46min$ (2fps) + $22min$ (1fps) + $11,5min$ (0,5fps) + $4,7min$ (TransNetV2)). Der Zeitaufwand bei zwei Basismodellen (`datacomp_l` + `datacomp_xl`) und zwei Fine-Tuning-Strategien beträgt circa 62 Stunden für die Evaluation, ohne Berücksichtigung der Trainingsdauer oder des OOD-Tests. Die Trainingszeit beim vollen Fine-Tuning beträgt circa 32 min und für das partielle Fine-Tuning circa 18 min.

4.6 Benutzeroberfläche

Die Benutzeroberfläche dient kompakt der Veranschaulichung der Funktionalität. Sie stellt ein ergänzendes Modul dar und ist kein Bestandteil der empirischen Untersuchungen.

Der Aufbau der Benutzeroberfläche besitzt zwei Modi. Der erste Modus bietet die Einzelsuche und der zweite einen Vergleichsmodus, der den Vergleich zweier Modelle miteinander ermöglicht. Zur Modellauswahl stehen der Modus als Radio-Button, die Fine-Tuning-Strategie, der α-Wert über einen Schieberegler und die Frame-Strategie zur Verfügung. Aus dieser Auswahl wird der Modell-Tag gebildet, welcher die Indizes adressiert. Das ermöglicht der App ohne eigene Logik auf die vorgerechneten Indizes zuzugreifen.
Ist ein Modell geladen, wird es im Cache zusammen mit den Indizes gespeichert, um eine erneute Ladezeit nach jeder einzelnen Anfrage zu verhindern.
Wurde ein Aufruf verarbeitet, werden die Top-K Ergebnisse als Bild mit Videokennung und Ähnlichkeitswert angezeigt.

Eine zu benennende Einschränkung ist, dass die Oberfläche ausschließlich den Test-Split durchsucht, da sie die Indizes der Evaluation nutzt und diese nur den Test-Split umfassen. Abbildung 4.1 zeigt den Aufbau der App im Vergleichsmodus.

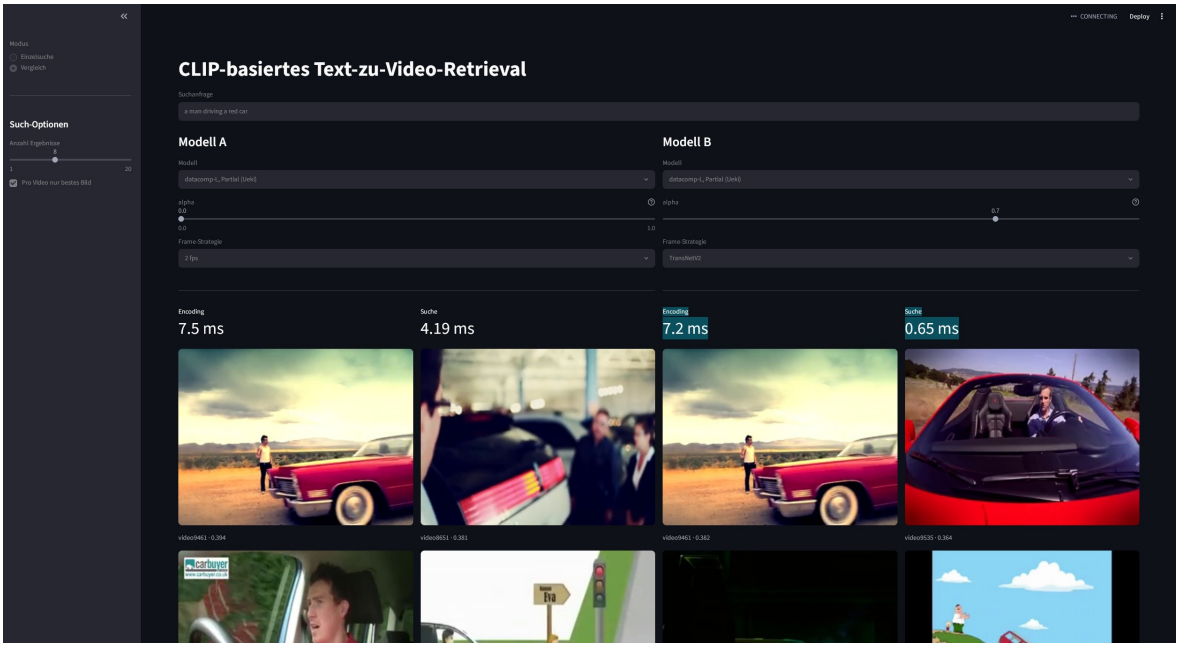


Abbildung 4.1: Screenshot von der Benutzeroberfläche der Streamlit-App vom 26.07.2026

4.7 Anpassungen während der Implementierung

Während der Implementation wurden Anpassungen getätigt, die wichtige Designentscheidungen zur Folge hatten:

Beim MSR-VTT sind verschiedene Split-Konventionen verbreitet und wurden auch in dem verwendeten Download mitgeliefert. Zum Trainieren wurde, wie unter Punkt 3.2.1 angekündigt, der offizielle Train und Val-Split verwendet, der als (`train_list_full.txt`) und (`val_list_full.txt`) verfügbar ist. Um die Arbeit vergleichbar mit anderen Papern

zu gestalten, wurde überlegt, neben dem offiziellen Test-Split, den in der Literatur teilweise verwendeten 1K-A Split zu untersuchen, um für die Ergebnisse eine bessere Vergleichbarkeit zwischen andernen Papern zu erzielen. Nach der Untersuchung der Split-Aufteilung wurde jedoch festgestellt, dass Teile des 1K-A Splits auch in dem verwendeten Trainings-Split zu finden sind, was die Evaluation verfälscht hätte.

Eine weitere Anpassung betraf die Wahl der Basismodelle des OpenCLIP ViT-B/16. Die erste Untersuchung wurde auf dem `datacomp_l` Pretraining-Checkpoint, ohne dessen Eignung vorab zu prüfen, ausgeführt. Bei späteren Recherchen wurde ermittelt, dass es durchaus geeignetere Checkpoints für diese Architektur gibt. Zu diesem Zeitpunkt war der Großteil der Evaluationen zu dem Modell bereits durchgeführt. Daraufhin wurde beschlossen, das Training des `datacomp_l` Checkpoints zu beenden und einen weiteren Lauf mit einem noch intensiver vortrainiertem Checkpoint, dem `datacomp_xl`, durchzuführen. Dadurch wurde das Spektrum der Ergebnisse erweitert und zusätzlich konnte untersucht werden, ob ein stärker vortrainiertes Modell andere Eigenschaften, als ein weniger vortrainiertes Modell, zeigt. Dies erweitert die Aussagekraft der Ergebnisse und lässt zusätzlich weitere Erkenntnisse zu, die im folgenden Kapitel 5 vorgestellt werden.

Kapitel 5: Ergebnisse und Auswertung

5.1 Überblick und Darstellungsweise

Im folgenden Kapitel werden mithilfe von Tabellen und Grafiken die Ergebnisse und Erkenntnisse dieser Arbeit präsentiert und interpretiert. Wie bereits in den vorherigen Kapiteln erwähnt, wurden 176 Messungen auf dem MSR-VTT-Datensatz über zwei Modellcheckpoints, zwei Fine-Tuning-Strategien, vier Frame-Strategien und elf α-Werte durchgeführt. Dazu kommen 22 weitere für den OOD-Test mit Charades. Aufgrund des Umfangs werden in diesem Kapitel nur Ausschnitte mit den aussagekräftigsten Werten der gesamten Tabellen und Grafiken gezeigt. Die vollständigen Messwerte befinden sich im Anhang.

Dazu werden primär folgende in Abschnitt 3.2.2 erklärten Metriken verwendet:

- **Recall@K** für die Anwenderperspektive aus dem VBS
- **Median-Rank** als robustes Maß
- **Mean-Rank** als Vergleichswert

Ein besonderes Auge wird auf die Recall@1-Werte gelegt, da beim KIS das erste Ergebnis für die Nutzer am kritischsten ist und die Treffersicherheit des Systems zeigt, da ohne manuelles Suchen direkt das richtige Ergebnis angezeigt wird.

5.2 Fine-Tuning

Zuerst wird die Wirkung des Fine-Tunings untersucht. Schnell wird erkennbar, dass trotz der niedrigen Lernrate und Batchsize eine deutliche Verbesserung über alle Metriken und Konfigurationen konsistent ist, vgl. Tabelle 5.1. Bei den unterschiedlichen Strategien (partiellen Fine-Tuning gegen volles Fine-Tuning vgl. 3.3.3), liegt das volle Fine-Tuning knapp vorne. Der größte Sprung ist nicht bei den Recall@K-Werten zu finden, wo die Werte nur um einzahlige Prozentpunkte steigen, sondern beim Median und Mean-Rank. Beim Medianrang ist bei beiden Basismodellen eine Verbesserung von mindestens 30% zu beobachten. Beim Mean-Rank kommt man sogar auf eine Verbesserung von rund 38% beim `datacomp_l` Basismodell und beim `datacomp_xl` Basismodell eine Verbesserung von rund 39%.

Beachtenswert ist, dass zwar das partielle Fine-Tuning ein klein wenig schlechter performt, es jedoch nur knapp 55% der Zeit zum Trainieren benötigte. Sollte ein zeitlicher Aspekt bei der Wahl im Vordergrund stehen, ist das partielle Fine-Tuning eine deutlich bessere Wahl.

Tabelle 5.1: Wirkung des Fine-Tunings bei 1fps auf dem MSR-VTT.

Konfiguration	**R@1**	**R@5**	**R@10**	**MedianR**	**MeanR**	**Trainingszeit (min)**
`datacomp_l`						
Zero-Shot	0,1592	0,3455	0,4338	16,0	162,94	–
Partiell	0,1819	0,3926	0,4876	**11,0**	106,14	18
Vollständig	**0,1923**	**0,4003**	**0,4983**	**11,0**	**101,01**	33
`datacomp_xl`						
Zero-Shot	0,2161	0,4120	0,5037	10,0	127,21	–
Partiell	0,2278	0,4522	0,5445	8,0	80,37	17
Vollständig	**0,2365**	**0,4629**	**0,5548**	**7,0**	**77,63**	32

5.3 Interpolationsfaktor

Die Bestimmung eines optimalen Interpolationsfaktors α erweist sich als komplex, da die Ergebnisse je nach Modellgröße, Fine-Tuning-Strategie und evaluierter Metrik variieren (siehe Tabelle 5.2). Dennoch ist ein wichtiger Befund sofort erkennbar: anders als in der Literatur, die einen α-Wert von 0,5 verwendet [24], liegt das Performance-Optimum in dem untersuchten System bei der Mehrheit der Konfigurationen deutlich höher im Bereich von $\alpha = 0,7$ bis $\alpha = 0,9$.

Der Mean Rank nimmt eine Sonderrolle ein, da dieser sein Optimum konstant bei $\alpha = 1,0$, also bei reinem Fine-Tuning, erreicht. Der Unterschied zu den Recall@K Metriken erklärt sich damit, dass diese nur den Kopf der Rangliste darstellen. Folglich ergibt sich, dass wenn das Zero-Shot-Modell für einen kleinen Teil der Ränge sehr schlechte Ergebnisse erzeugt, zieht dies den Mean-Rank deutlich nach oben. Jede Gewichtsinterpolation mischt das Zero-Shot-Modell mit in seine Gewichte und das resultierende Modell verhält sich aufgrund dessen teilweise wie das Zero-Shot-Modell. Dieses Verhalten scheint am Verteilungsende stärkere Auswirkungen zu zeigen als am Kopf der Rangliste. Somit werden vor allem die für den VBS wichtigen oberen Ränge durch das Interpolieren verbessert. Der Median-Rank ist zudem nicht so betroffen wie der Mean-Rank, was Hinweise darauf gibt, dass das Problem bei einzelnen Ausreißern liegt.

Die absoluten Gewinne gegenüber dem reinen Fine-Tuning fallen gering aus. Die mittlere Differenz zum reinen Fine-Tuning über alle vier Frame-Strategien und beide Fine-Tuning-Strategien pro Modellcheckpoint sind in Tabelle 5.6 dargestellt.

Tabelle 5.2: Lage des optimalen Interpolationsfaktors α je Konfiguration und Metrik.

Modell	**Frame-Strategie**	**R@1**	**R@5**	**R@10**	**Median Rank**	**Mean Rank**
`datacomp_l`						
Partiell	2fps	0,8	0,9	0,7	0,7*	1,0
	1fps	0,7	1,0	0,9	0,6*	1,0
	0,5fps	0,7	0,9	0,8	0,6*	1,0
	TransNetV2	0,8	0,7*	0,8*	0,5*	1,0
Vollständig	2 fps	0,8*	0,8*	0,8	0,6*	1,0
	1fps	1,0	0,9	0,8*	0,6*	1,0
	0,5fps	0,8	0,9	0,9	0,7*	1,0
	TransNetV2	0,8	0,8	0,9	0,6*	1,0
`datacomp_xl`						
Partiell	2fps	0,8	0,7	1,0	0,8	1,0
	1fps	0,8	0,8	0,7*	0,7	1,0
	0,5fps	0,8	0,7	1,0	0,4*	1,0
	TransNetV2	0,9	0,8*	0,8	0,5*	1,0
Vollständig	2 fps	1,0	1,0	1,0	0,7*	1,0
	1fps	0,9	1,0	1,0	0,7*	1,0
	0,5fps	0,7	1,0	1,0	0,8*	1,0
	TransNetV2	0,7	0,9	1,0	0,9*	1,0

* mehrere α liefern denselben Wert; angegeben ist das kleinste.

Bei $\alpha = 0,8$ liegen die durchschnittlichen Verbesserungen bei R@1 bei 0,13 bei `datacomp_l` und bei 0,33 Prozentpunkten bei `datacomp_xl`. Diese geringen Abstände lassen sich mit dem WiSE-FT Paper [27] erklären. So zeigen sich laut diesem Paper zwar geringe Gewinne bei In-Domain-Performance, aber die großen Gewinne treten vor allem unter Distribution-Shift auf. Aus diesem Grund wurde der in Abschnitt 5.6 evaluierte OOD-Test durchgeführt.

Auch wenn einzelne Differenzen sehr gering sind, zeigt sich, dass bei 14 von 16 das R@1-Optimum nicht bei $\alpha = 1,0$, sondern im Bereich von $\alpha = 0,7 - 0,9$ liegt. Da alle Konfigurationen mit denselben Anfragen durchgeführt wurden, lassen sich diese Unterschiede direkt miteinander vergleichen. Auch bei der Kurvendarstellung von R@1 in Abbildung 5.1 ist erkennbar, dass diese unabhängig von Modell und Strategie alle gleichmäßig bis circa $\alpha = 0,7$ steigen, dann abflachen und nach ihrem Optimum sinken. Dieses Bild zeigt sich ebenfalls bei den anderen R@5 und R@10 Werten (vgl. 5.2).

Ein universeller perfekter Interpolationsfaktor existiert mit Hinblick auf die vorhandenen Ergebnisse nicht, da dieser stark mit den jeweiligen Konfigurationen und Basismodellen variiert. Da die Forschungsfrage jedoch nach *einem* optimalen Faktor verlangt, empfiehlt sich für die vorliegende Domäne im Kontext des VBS ein Faktor von 0,8. Dieser entspricht nicht nur dem Median über alle optimalen R@1 sowie alle zusammengenommenen Recall-Werte, sondern deckt sich auch mit dem Durchschnitt, welcher bei R@1 $\approx 0,81$ und bei allen Recall-Werten zusammen $\approx 0,85$ beträgt. Die Empfehlung stützt sich auf die Recall-

Tabelle 5.3: WiSE-FT Alpha-Sweep für `datacomp_l` (1fps).

	partial				full			
α	**R@1**	**R@5**	**R@10**	**MnR**	**R@1**	**R@5**	**R@10**	**MnR**
0,0	0,1592	0,3455	0,4338	162,9	0,1592	0,3455	0,4338	162,9
0,1	0,1642	0,3532	0,4428	153,9	0,1645	0,3535	0,4448	152,3
0,2	0,1686	0,3592	0,4555	145,1	0,1692	0,3605	0,4605	142,4
0,3	0,1736	0,3625	0,4652	136,8	0,1739	0,3656	0,4726	133,3
0,4	0,1773	0,3679	0,4756	129,4	0,1783	0,3729	0,4816	125,5
0,5	0,1816	0,3793	0,4826	123,0	0,1856	0,3819	0,4860	118,9
0,6	0,1829	0,3853	0,4870	117,2	0,1833	0,3940	0,4883	113,3
0,7	**0,1860**	0,3896	0,4910	112,4	0,1856	0,3977	0,4957	108,6
0,8	0,1826	0,3896	0,4936	109,0	0,1893	0,4013	**0,4983**	104,9
0,9	0,1816	0,3906	**0,4943**	106,9	0,1880	**0,4020**	0,4980	102,3
1,0	0,1819	**0,3926**	0,4876	**106,1**	**0,1923**	0,4003	**0,4983**	**101,0**

Tabelle 5.4: WiSE-FT Alpha-Sweep für `datacomp_xl` (1fps).

	patial				full			
α	**R@1**	**R@5**	**R@10**	**MnR**	**R@1**	**R@5**	**R@10**	**MnR**
0,0	0,2161	0,4120	0,5037	127,2	0,2161	0,4120	0,5037	127,2
0,1	0,2227	0,4181	0,5151	118,9	0,2207	0,4157	0,5171	118,5
0,2	0,2247	0,4244	0,5244	111,0	0,2254	0,4217	0,5278	110,2
0,3	0,2271	0,4321	0,5308	103,7	0,2294	0,4328	0,5328	102,9
0,4	0,2278	0,4355	0,5331	97,2	0,2311	0,4391	0,5358	96,5
0,5	0,2314	0,4401	0,5415	91,9	0,2321	0,4452	0,5425	91,1
0,6	0,2331	0,4438	0,5462	87,7	0,2324	0,4515	0,5462	86,6
0,7	0,2351	0,4468	**0,5488**	84,4	0,2365	0,4542	0,5482	83,2
0,8	**0,2375**	**0,4528**	**0,5488**	82,1	0,2395	0,4582	0,5515	80,5
0,9	0,2331	0,4525	0,5448	80,7	**0,2401**	0,4609	0,5528	78,7
1,0	0,2278	0,4522	0,5445	**80,4**	0,2365	**0,4629**	**0,5548**	**77,6**

Tabelle 5.5: R@1 in Abhängigkeit von α für über alle Frame-Strategien (Full-FT-Variante beider Modelle).

	`datacomp_l`				`datacomp_xl`			
α	**0,5fps**	**1fps**	**2fps**	**TransNetV2**	**0,5fps**	**1fps**	**2fps**	**TransNetV2**
0,0	0,1572	0,1592	0,1602	0,1458	0,2074	0,2161	0,2137	0,1766
0,1	0,1639	0,1645	0,1652	0,1478	0,2124	0,2207	0,2174	0,1799
0,2	0,1676	0,1692	0,1682	0,1488	0,2174	0,2254	0,2194	0,1843
0,3	0,1712	0,1739	0,1709	0,1472	0,2201	0,2294	0,2221	0,1883
0,4	0,1759	0,1783	0,1742	0,1505	0,2207	0,2311	0,2264	0,1957
0,5	0,1789	0,1856	0,1779	0,1542	0,2227	0,2321	0,2304	0,1973
0,6	0,1786	0,1833	0,1799	0,1582	0,2231	0,2324	0,2331	0,2017
0,7	0,1786	0,1856	0,1836	0,1622	**0,2284**	0,2365	0,2331	**0,2067**
0,8	**0,1799**	0,1893	**0,1876**	**0,1629**	0,2261	0,2395	0,2321	0,2057
0,9	0,1793	0,1880	**0,1876**	0,1619	0,2261	**0,2401**	0,2338	0,2023
1,0	0,1786	**0,1923**	0,1853	0,1602	0,2278	0,2365	**0,2351**	0,2030

Tabelle 5.6: Mittlere Differenz zum reinen Fine-Tuning ($\alpha = 1$), gemittelt über alle acht Konfigurationen je Modell. (Formatiert und berechnet mit Claude Opus 5)

	`datacomp_l`				`datacomp_xl`			
α	**ΔR@1**	**ΔR@5**	**ΔR@10**	**ΔMnR**	**ΔR@1**	**ΔR@5**	**ΔR@10**	**ΔMnR**
0,0	-2,01	-4,81	-5,35	+59,47	-1,96	-4,30	-4,59	+49,15
0,1	-1,57	-4,00	-4,56	+49,45	-1,53	-3,61	-3,50	+40,51
0,2	-1,26	-3,29	-3,41	+39,93	-1,17	-2,94	-2,63	+32,37
0,3	-1,06	-2,58	-2,47	+31,09	-0,89	-2,05	-2,09	+24,81
0,4	-0,68	-1,95	-1,69	+23,32	-0,64	-1,43	-1,62	+18,23
0,5	-0,32	-1,22	-1,06	+16,71	-0,27	-0,93	-1,03	+12,71
0,6	-0,20	-0,63	-0,55	+11,16	-0,03	-0,56	-0,73	+8,26
0,7	+0,03	-0,14	-0,18	+6,64	+0,25	-0,30	-0,41	+4,78
0,8	**+0,13**	+0,04	+0,17	+3,28	**+0,33**	-0,12	-0,01	+2,27
0,9	+0,05	**+0,16**	**+0,21**	+1,07	+0,24	-0,01	-0,05	+0,69
1,0	+0,00	+0,00	+0,00	**+0,00**	+0,00	**+0,00**	**+0,00**	**+0,00**

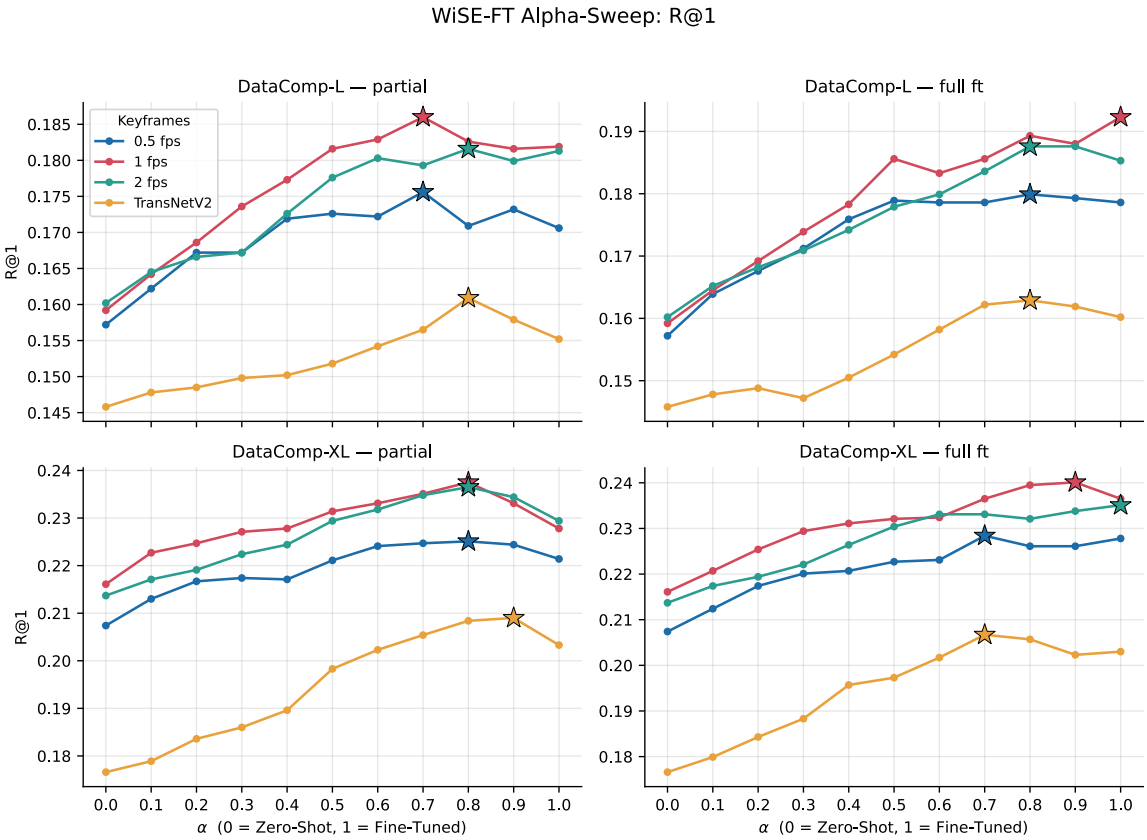


Abbildung 5.1: Recall@1 über alle α, unterteilt in Modell und Fine-Tuning-Strategie

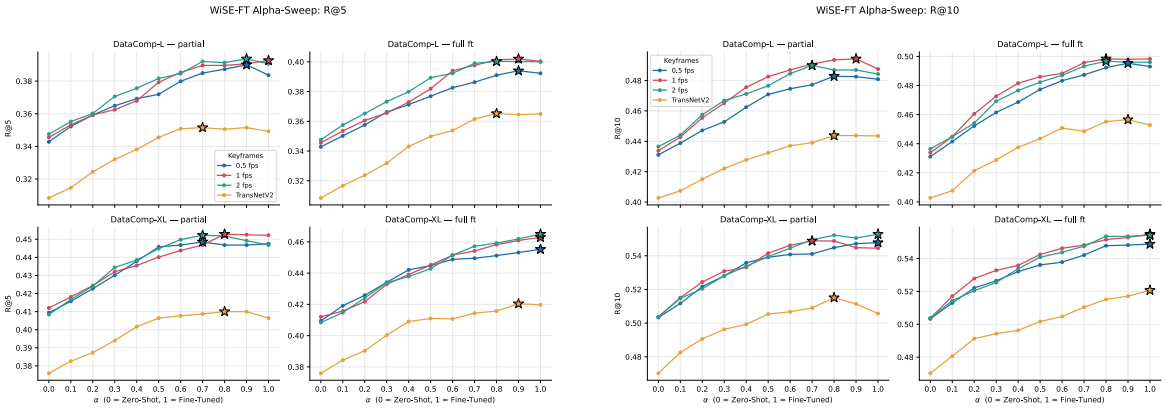


Abbildung 5.2: Recall@5 und Recall@10 über alle α, unterteilt in Modell und Fine-Tuning-Strategie

Metriken, da diese im VBS-Kontext wichtiger sind als der Median oder Mean-Rank. Würde man sich nur auf die mittlere Rangposition beziehen, wäre reines Fine-Tuning vorzuziehen, da es über alle Tests die besseren Ergebnisse liefert.

5.4 Frame-Sampling

Die zweite Forschungsfrage nach der besseren Frame-Sampling-Strategie lässt sich um einiges simpler beantworten als die Frage nach dem Interpolationsfaktor.

Tabelle 5.7 zeigt, dass 1fps der Sweet Spot bei den Frame-Sampling Strategien ist. So zeigt sich sowohl beim `datacomp_l` als auch beim `datacomp_xl`, dass die besten Recall@1 und Recall@10 Werte bei 1fps liegen. Außerdem fällt auf, dass, trotz der doppelten Abtastrate, 2fps keinen Leistungsgewinn bringt, sondern sogar in manchen Metriken einen leichten Abfall zeigt (R@1 fällt bei `datacomp_l` relativ um 2,44 %). Im Vergleich zu der niedrigeren Abtastrate von 0,5fps schlägt 1fps diese vor allem bei den R@1 Werten (bei `datacomp_l` um 6,45% und bei `datacomp_xl` um 4,87%). Dieser Vorteil flacht jedoch bei dem höheren R@5 und R@10-Werten auf unter 2% ab.

Das szenenbasierte Sampling mit TransNetV2 erzielt in jeglichen untersuchten Metriken und Checkpoints deutlich schlechtere Ergebnisse, als die zeitbasierten Varianten. Bei R@1 beim `datacomp_l` liegt TransNetV2 über 15% hinter dem 1fps Sampling. Und selbst beim Mean Rank, bei welchem die verschiedenen zeitbasierten Methoden kaum Abstand zueinander zeigten, verschlechtert sich dieser um knapp 20%.

Tabelle 5.7: Beste erreichte Werte je Frame-Strategie

Strategie	**R@1**	**R@5**	**R@10**	**MeanR**	**MedianR**
`datacomp_l`					
2fps	0,1876	0,4003	0,4967	**100,76**	**11**
1fps	**0,1923**	**0,4020**	**0,4983**	101,01	**11**
0,5fps	0,1799	0,3940	0,4953	101,81	**11**
TransNetV2	0,1629	0,3652	0,4565	121,18	14
`datacomp_xl`					
2fps	0,2365	**0,4649**	0,5542	78,38	**7**
1fps	**0,2401**	0,4629	**0,5548**	77,63	**7**
0,5fps	0,2284	0,4552	0,5488	**76,93**	8
TransNetV2	0,2090	0,4204	0,5207	94,15	9

Da der Fokus der Forschungsfrage neben der Performance auch auf einem verhältnismäßigen Speicheraufwand liegt, muss der Speicheraufwand und die Erstellungsdauer ebenso aufgezeigt werden. Tabelle 5.8 zeigt die Anzahl der Frames im Index und deren mittlere Indexierungsdauer mit Performancevergleich zu R@1 beim `datacomp_xl`. Hier fällt auf, dass sowohl die Indexierungsdauer als auch der Speicherbedarf linear mit der Frame-Strategie

skalieren.
Auch hier ist offensichtlich, dass das 2fps Sampling vollkommen ineffizient ist. Es nutzt nicht nur mehr als die doppelte Menge an Indizes und braucht die doppelte Indizierungsdauer, sondern es liefert auch die schlechtere Performance und schneidet somit klar am schlechtesten im Verhältnis von Speicher zur Performance ab.
Bei 0,5fps fällt die Bewertung etwas schwieriger. So benötigt diese Strategie nur etwa die Hälfte des Speichers und der Indexierungsdauer im Verhältnis zu 1fps, es bleiben aber über 95% der R@1 Performance erhalten, bei R@5 und R@10 sogar mehr. Das macht diese Strategie bei platz- und zeitlimitierten Anwendungen sehr interessant bezüglich der Effizienz.
TransNetV2 ist zwar noch leichtgewichtiger als 0,5fps, bringt aber auch einen klaren Qualitätseinbruch (mind. 12,95% von 1fps R@1) mit sich. Folglich eignet sich diese Strategie nur, wenn die Ressourcen extrem eingeschränkt sein sollten.

Zudem muss erwähnt werden, dass der Speicheraufwand vor allem vor der Generierung der Indizes ein Argument ist. Nach der Indizierung sinkt der Speicheraufwand extrem und der Unterschied zwischen 2fps und TransNetV2 beträgt nur noch 167 MiB (vgl. 4.5). Allerdings skaliert auch dieser linear und gerade bei Wettbewerben wie dem VBS, sind die Datenmengen um ein Vielfaches größer als der Test-Split des MSR-VTT. Als Limitation bleibt zu erwähnen, dass die szenenbasierte Sampling-Strategie auf längere Videos ausgelegt ist und hier mit nur 3,4 Frames pro Video im Schnitt zu wenig Datenpunkte geliefert hat, um wirklich konkurrenzfähig zu sein. In einem Szenario mit längeren Videos könnte diese Strategie deutlich besser abschneiden.

Tabelle 5.8: Anzahl der Frames im Index und mittlere Indexierungsdauer mit Performancevergleich zu R@1

Strategie	**Frames im Index**	**Indexierungsdauer**	**Relative Datenmenge**	**R@1 (`datacomp_xl`)**	**Performance-Erhalt**
2fps	95.729	46,0 min	205,0%	0,2365	98,5%
1fps	46.708	22,0 min	100,0%	0,2401	100,0%
0,5fps	24.074	11,5 min	51,5%	0,2284	95,1%
TransNetV2	10.064	4,7 min	21,5%	0,2090	87,0%

Bei der Wahl der Frame-Sampling-Strategie erweist sich die 1fps-Strategie als das beste Gesamtsystem, da es eindeutig die beste Retrieval Qualität und Performance bei einer moderaten Indizierungsdauer mit 22 Minuten bietet. Eine Erhöhung auf 2fps verdoppelt den Speicher- und Zeitaufwand ohne jeglichen Leistungsgewinn und kann nicht empfohlen werden.
Demgegenüber stellt die 0,5fps Strategie eine sehr effiziente Strategie dar, die bei einer Halbierung des Speicher- und Zeitaufwandes über 95% der Performance erhält.
TransNetV2 liefert zwar die mit Abstand geringste Indizierungszeit und auch Speicheraufwand, muss dafür aber einen spürbaren Performanceverlust hinnehmen. Somit liefert TransNetV2 zwar die beste Effizienz, im VBS-Kontext ist jedoch die Treffergenauigkeit

wichtiger als der Speicheraufwand, weshalb die 1fps Strategie die folgerichtige Wahl bei der vorliegenden Fragestellung ist.

5.5 Einfluss der Basismodelle

Der Vergleich der beiden Basismodelle verläuft wie erwartet. Der mit deutlich mehr Daten vortrainierte `datacomp_xl` Pretraining-Checkpoint liefert über alle Messungen und Strategien, trotz identischem Training, einen deutlichen Performance-Vorsprung im Gegensatz zum `datacomp_l` Pretraining-Checkpoint. Keine einzige Metrik favorisiert das weniger vortrainierte Modell, wie Tabelle 5.9 zeigt. Die Sprünge sind deutlich: R@1 von 19,3% zu 24% bei 1fps, bester Median-Rank bei 1fps von 11 auf 7, usw. Diese Ergebnisse machen den Vorteil eines besser vortrainierten Modells deutlich.

Tabelle 5.9: Vergleich der `datacomp_l`- und `datacomp_xl`-Modellen bei 1fps.

Modell-Konfiguration	Modell	R@1	R@5	R@10	MedianR	MeanR
Basismodell	`l`	0,1592	0,3455	0,4338	16,0	162,92
	`xl`	**0,2161**	**0,4120**	**0,5037**	**10,0**	**127,21**
Partial Ep7	`l`	0,1819	0,3926	0,4876	11,0	106,14
	`xl`	**0,2278**	**0,4522**	**0,5445**	**8,0**	**80,37**
Full Ep7/6	`l`	0,1923	0,4003	0,4983	11,0	101,01
	`xl`	**0,2365**	**0,4629**	**0,5548**	**7,0**	**77,63**
Best WiSE-FT Partial	`l` ($\alpha = 0,7$)	0,1860	0,3896	0,4910	11,0	112,44
	`xl` ($\alpha = 0,8$)	**0,2375**	**0,4528**	**0,5488**	**8,0**	**82,06**
Best WiSE-FT Full	`l` ($\alpha = 1,0$)	0,1923	0,4003	0,4983	11,0	101,01
	`xl` ($\alpha = 0,9$)	**0,2401**	**0,4609**	**0,5528**	**7,0**	**78,68**

Bei genauerer Beobachtung fällt auf, dass die Wahl des Basismodells einen größeren Einfluss auf die Ergebnisse hat, als die untersuchte Optimierung des Modells. Beispielsweise erreicht der unveränderte `datacomp_xl`-Checkpoint einen R@1-Wert von 0,2161 und damit einen besseren Wert als die beste Konfiguration des `datacomp_l`-Modells bei vollem Fine-Tuning und 1fps mit R@1 = 0,1923. Dieser Unterschied ist bei den anderen Metriken nicht so extrem, zeigt aber einen klaren Trend.

Ebenso wird ersichtlich, dass bei der Epochenwahl aus Abschnitt 4.3.3, das stärkere Basismodell beim Fine-Tuning, mit exakt gleichen Bedingungen, früher sein Optimum erreicht. Das deutet darauf hin, dass stärker vortrainierte Modelle weniger Anpassung benötigen als weniger vortrainierte Modelle.

Es sei erneut darauf hinzuweisen, dass beide Modelle mit derselben, durch das Warmup niedrigeren Lernrate trainiert wurden. Das macht den der Arbeit zugrunde liegenden Vergleich zwar intern gültig 4.3.2, jedoch müsste eine Untersuchung unter optimalen Trainingsbedingungen für eine allgemein gültige Aussage ausgeführt werden. Außerdem basieren beide

untersuchten Modelle auf derselben Architektur und denselben Pretraining-Verfahren, somit müsste auch hier ein Vergleich mit weiteren Architekturen durchgeführt werden, um universelle Aussagen zu treffen.

5.6 Verhalten unter Distribution-Shift

Um die Generalisierungsfähigkeit der entwickelten Verfahren zu evaluieren, wurde die Leistungsfähigkeit des Modells unter einem Distribution-Shift untersucht. Der Versuchsaufbau bleibt identisch zu den vorherigen Tests. Hierbei kommt das `datacomp_xl`-Modell mit der 1fps Frame-Strategie zum Einsatz, da diese Kombination die besten Ergebnisse in den vorliegenden Messungen lieferte. Nur der Datensatz wird angepasst.

Auffallend bei den Ergebnissen in Tabelle 5.10 ist, dass das absolute Leistungsniveau deutlich unter den In-Distribution-Ergebnissen des MSR-VTT liegt. Während der R@1 zuvor bei einem Bestwert von $0,24$ lag, sinkt der Bestwert auf dem Charades-Datensatz auf $0,074$. Diese Absenkung bedeutet nicht, dass das Evaluationssystem fehlerhaft funktioniert. Vielmehr ist dieses Ergebnis auf den Datensatz zurückzuführen. Die enthaltenen Videos zeigen alle visuell und thematisch sehr ähnliche Alltagshandlungen in vergleichbaren Umgebungen 3.2.1, wodurch die Unterscheidbarkeit einzelner Clips deutlich sinkt. Außerdem ist hinzuzufügen, dass die Captions, im Gegensatz zum MSR-VTT, erheblich länger sind.

Tabelle 5.10: Alpha-Sweep auf Charades mit `datacomp_xl` und 1fps.

	partielles Fine-Tuning					vollständiges Fine-Tuning				
α	**R@1**	**R@5**	**R@10**	**MedR**	**MeanR**	**R@1**	**R@5**	**R@10**	**MedR**	**MeanR**
0,0	0,0682	0,1744	0,2469	50,0	183,94	0,0682	0,1744	0,2469	**50,0**	183,94
0,1	0,0709	0,1787	0,2566	**49,0**	182,47	0,0714	0,1777	0,2534	**50,0**	182,44
0,2	0,0709	0,1825	0,2582	**49,0**	181,35	**0,0746**	**0,1830**	0,2555	**50,0**	181,20
0,3	0,0719	**0,1830**	0,2593	**49,0**	180,47	0,0725	0,1793	0,2491	**50,0**	180,20
0,4	**0,0735**	0,1809	0,2625	**49,0**	179,90	**0,0746**	0,1804	0,2560	51,0	179,34
0,5	0,0709	0,1793	0,2609	50,0	**179,73**	**0,0746**	0,1782	**0,2603**	52,0	179,14
0,6	0,0730	0,1782	**0,2630**	51,0	180,04	0,0709	0,1809	0,2566	53,0	**179,04**
0,7	0,0719	0,1755	0,2576	53,0	180,93	0,0698	0,1804	0,2566	53,0	179,27
0,8	0,0692	0,1771	0,2507	54,0	182,35	0,0660	0,1804	0,2550	55,0	179,66
0,9	0,0666	0,1755	0,2426	56,0	184,74	0,0666	0,1777	0,2469	56,0	180,33
1,0	0,0617	0,1739	0,2389	57,0	187,87	0,0639	0,1744	0,2437	57,0	181,45

Die erste wichtige Erkenntnis unter Distribution-Shift lautet, dass hier reines Fine-Tuning schlechter als das Zero-Shot-Modell abschneidet. Der R@1-Wert verschlechtert sich von 6,82% auf 6,17% (partielles Fine-Tuning) bzw. 6,39% (vollständiges Fine-Tuning). Noch größere Abstriche müssen beim Median-Rank gemacht werden, dieser steigt von ursprünglich Rang 50 auf Rang 57 bei beiden Fine-Tuning-Strategien. Folgend leidet das Modell,

wegen der Spezialisierung auf die MSR-VTT Daten, unter dem Verlust seiner Generalisierungsfähigkeit.

Durch den Einsatz der Interpolation kann der Leistungsabfall verhindert werden und es wird sogar eine Verbesserung zu dem Zero-Shot-Modell erzielt. Das Optimum des Interpolationsfaktors verschiebt sich allerdings im Gegensatz zum In-Distribution-Test deutlich nach unten. So liegt das optimale α zwischen 0,2 und 0,6. Für die Top-Ranking-Performance bei R@1 und R@5 sogar nur zwischen 0,2 und 0,4. So steigt gegenüber dem reinen Fine-Tuning R@1 um 1,18 Prozentpunkte beim partiellen Fine-Tuning und 1,07 Prozentpunkte beim vollständigen. Auch das Niveau des Zero-Shot-Modells wird dabei leicht um 0,52 bzw. 0,64 Prozentpunkte übertroffen. Das deutet darauf hin, dass unter starkem Distribution-Shift ein höherer Anteil des ursprünglichen Zero-Shot-Modells beibehalten werden muss, um optimale Werte zu erhalten.

Vergleicht man beide Fine-Tuning-Strategien fällt auf, dass kaum Unterschiede zwischen dem partiell- und voll-feingetunten Modellen unter Distribution-Shift bestehen. Das aufwändigere Fine-Tuning lohnt sich also unter Distribution-Shift nicht, da beide Verfahren über das gesamte α-Spektrum hinweg eine sehr ähnliche Dynamik und Bestwerte erreichen, siehe 5.10.

Bei der Interpretation der Ergebnisse muss berücksichtigt werden, dass nur eine Frame-Sampling-Strategie und ein ausgewähltes Basismodell beim OOD-Test untersucht wurde. Außerdem wurde lediglich eine zufällig gewählte Textbeschreibung gewählt, die pro Video stark in Länge und Genauigkeit variiert. Um die Aussage zu verschärfen, müsste ein weiterer OOD-Datensatz untersucht werden. Dabei wäre zu beachten, dass Charades ein sehr spezifischer Datensatz ist und ein anderer Shift, wie z.B. in Richtung Sport, eine andere Verschiebung der Ergebnisse zur Folge haben könnte.

5.7 Zusammenfassende Bewertung

Betrachtet man die Wirksamkeit aller untersuchten Parameter, ergibt sich eine klare Rangfolge. Die Wahl des Basismodells zeigte den größten Einfluss auf das Endresultat, gefolgt von der Frame-Sampling-Strategie oder dem Fine-Tuning, je nachdem, ob TransNetV2 mitberechnet oder weggelassen wurde. Der im Zentrum der ersten Forschungsfrage stehende Interpolationsfaktor wiegt am schwächsten bei den vorliegenden Untersuchungen, ein klarer Vorteil konnte lediglich bei dem OOD-Test festgestellt werden. Den genauen Einfluss aller Strategien auf die R@1 Werte mit der Effektstärke sind in Tabelle 5.11 aufgelistet.

Der Einfluss der Fine-Tuning-Strategie erweist sich als nicht so groß wie vorher angenommen. So schlägt das vollständige Fine-Tuning zwar das partielle in-Distribution, aber nur um 0,87 Prozentpunkte beim R@1. Dafür benötigt das partielle Fine-Tuning jedoch nur 55% der Trainingszeit vom vollen Fine-Tuning, was gerade bei längeren Trainingssessions mit größeren Modellen und Datensätzen skalieren sollte. Außerdem profitiert das partielle Fine-Tuning stärker von der Gewichtsinterpolation(+0,97 vs. +0,36 Prozentpunkte bei

R@1). Aber trotzdem bleibt es unter der Performance des vollständigen Fine-Tunings. Unter Distribution-Shift verschwindet der Unterschied vollständig. Zusammenfassend kann die Aussage gemacht werden, dass vollständiges Fine-Tuning sich nur In-Domain knapp lohnt. Bei unbekannter Zieldomäne verschiebt sich die Erkenntnis dahin, dass partielles Fine-Tuning verwendet werden sollte.

Tabelle 5.11: Zusammenfassung der Befunde mit Effektstärke in Prozentpunkten. Effektangaben beziehen sich auf R@1 in Prozentpunkten für jeweils die beste Konfiguration.

Befund	**Effekt**
Basismodell	
`datacomp_xl` schlägt `datacomp_l` durchgängig	+4,78
Fine-Tuning	
Fine-Tuning verbessert alle Metriken gegenüber Zero-Shot	+2,04
Vollständiges Fine-Tuning schlägt partielles (in-Domain)	+0,87
Partielles Fine-Tuning benötigt rund 55% der Trainingszeit	—
Frame-Sampling	
1fps schlägt szenenbasiertes Sampling (TransNetV2)	+3,11
1fps schlägt 0,5fps	+1,17
2fps bringt gegenüber 1fps keinen Gewinn	−0,36
Interpolationsfaktor	
In-Domain liegt das optimale α bei 0,7 - 0,9	—
Gewinn der Interpolation gegenüber reinem Fine-Tuning (in-Domain)	+0,36 bis +0,97
Reines Fine-Tuning verschlechtert die OOD-Performance	−0,43 bis −0,65
Interpolation kompensiert den OOD-Verlust vollständig	+1,07 bis +1,18
OOD-Optimum verschiebt sich auf $\alpha = 0{,}2$ - 0,6	—
Kaum Unterschied zwischen den FT-Strategien unter Distribution-Shift	±0,37

Kapitel 6: Zusammenfassung und Ausblick

6.1 Zusammenfassung

Ausgangslage und Aufgabe

Die vorliegende Arbeit hat sich der Untersuchung des Problems der Videosuche über Texteingabe im Kontext der Known-Item-Search beim Video Browser Showdown angenommen. Zur Optimierung dieses Verfahrens wurden zwei Parameter untersucht, die in der Literatur bisher meist fest gewählt werden, anstatt diese empirisch zu prüfen.

Zum einen wurde untersucht, welche Frame-Sampling-Strategie die besten Ergebnisse im Hinblick auf Suchqualität mit vertretbarem Speicheraufwand bietet. Dazu wurden drei feste Abtastraten (2fps, 1fps, 0,5fps) und eine inhaltsbasierte Strategie (TransNetV2) verglichen. Zum anderen wurde geprüft, welcher von elf untersuchten Interpolationsfaktoren α (0,0; 0,1;0,2;...; 1,0), zwischen dem Zero-Shot-Modell und dem feingetunten Modell, die beste Retrieval-Qualität bietet und ob ein interpoliertes Modell beide Endpunkte übertreffen kann. Dabei wurden zwei Fine-Tuning-Strategien (partielles Fine-Tuning und vollständiges Fine-Tuning) und zwei Basismodelle (`datacomp_l` und `datacomp_xl`) untersucht und miteinander verglichen.

Vorgehen

Um die Untersuchung durchzuführen wurde ein Python Prototyp entwickelt, der von der Zerteilung der Videos in Keyframes bis hin zu Darstellung und Ranking der Suchergebnisse, die gesamte Pipeline der Videosuche über eine Texteingabe implementiert. Zur Evaluation fand ein systematischer Sweep über alle untersuchten Parameter statt, um nicht alle Eingaben händisch tätigen zu müssen. Das dient der Vermeidung von Fehlern und empfiehlt sich bei einem Umfang von knapp 200 Messungen aus zeitlichen Gründen. Dabei wurde darauf geachtet, dass während der Evaluation und bei jeder Messung identische Vergleichsbedingungen vorliegen.

Als letzter Schritt wurde noch ein Out-Of-Distribution-Test als ergänzende Untersuchung ausgeführt.

Kernergebnisse

Beim Frame-Sampling liefert die inhaltsbasierte Strategie durchgehend schlechtere Ergebnisse als die untersuchten festen Raten, verbraucht aber auch am wenigsten Speicherplatz. Die Unterschiede zwischen den festen Raten sind klein und metrikabhängig. Vor allem zwischen 2fps und 1fps zeigen sich nur minimale Unterschiede. Das 1fps-Sampling liegt mit nur der Hälfte des Speicherverbrauches und der Erstellungszeit gegenüber 2fps vorne und sollte aus diesem Grund bevorzugt werden. Die 0,5fps Variante punktet bezüglich der Sparsamkeit, da diese nur die Hälfte an Speicherplatz und Erstellungszeit im Vergleich zu 1fps bei rund 5% geringerem R@1 (bei R@5/R@10 ist der Abstand kleiner) benötigt.

Das Optimum des α-Wertes für die Interpolation liegt zwar in der Mehrzahl der Konfigurationen über dem in der Literatur fix gewählten Wert von $\alpha = 0,5$, variiert aber zwischen verglichener Metrik und Konfiguration. Im Durchschnitt und Median liefert bei dieser Untersuchung ein Wert von $\alpha = 0,8$ die besten Ergebnisse. Dies bedeutet, dass ein interpoliertes Modell nicht nur das Zero-Shot-Modell, sondern auch das feingetunte Modell unter den vorliegenden Bedingungen übertrifft.

Bei der Verwendung des Interpolationsfaktors sollte allerdings beachtet werden, dass die Gewinne in-Distribution klein und metrikabhängig sind. Zusätzlich wurde festgestellt, dass von den zwei verwendeten Fine-Tuning-Strategien das vollständige Fine-Tuning die besseren Ergebnisse liefert.

Der Out-Of-Distribution-Test zeigt andere Ergebnisse. Die auf dem MSR-VTT feingetunten Modelle schneiden in der vorliegenden Untersuchung schlechter ab, als die Zero-Shot Basismodelle. Die besten Ergebnisse liefern die interpolierten Modelle, das optimale α verschiebt sich deutlich nach unten.

Zur Einordnung der Ergebnisse muss ergänzt werden, dass die Wahl des Pretraining-Checkpoints einen höheren Einfluss auf das Ergebnis hatte, als die anderen in der Arbeit untersuchten Optimierungen.

Erbrachte Leistungen

Zur Bestimmung der Ergebnisse dieser Arbeit wurde ein Python Prototyp entwickelt, der dank seiner leichten Erweiterbarkeit und Modularität für weitere Untersuchungen angewendet werden kann. Er bietet zudem eine automatisierte und fortsetzbare Auswertung, die für eine große Anzahl an Messungen, wie sie in dieser Arbeit durchgeführt wurden, sinnvoll ist. Außerdem ist eine Datenbasis von rund 200 Messungen über zwei Basismodelle mit vier Frame-Sampling-Strategien, zwei Fine-Tuning-Strategien, elf α-Werten und 2 Datensätzen entstanden. Anhand dieser Messungen konnten neue Erkenntnisse bei der Wahl der Frame-Sampling-Strategie und dem Interpolationsfaktor gesammelt werden. Der Prototyp bietet zusätzlich eine interaktive Oberfläche, in der die textbasierte Videosuche

mit allen untersuchten Konfigurationen beim MSR-VTT-Datensatz interaktiv ausgeführt werden kann.

Grenzen

Diese Arbeit ist durch die Ausmaße einer Bachelorarbeit begrenzt. So wurde zum Beispiel nur ein Hauptdatensatz gewählt aus dem knapp 90% der Ergebnisse entstanden sind. Übertragbarkeit auf andere, größere Datensätze oder Domänen ist damit noch offen. Zudem wurde nur eine Modellarchitektur (ViT-B/16) untersucht, da diese durch die vorhandene Hardware begrenzt war. Die gemessenen Zeiten der Durchführung der Ergebnisse sind nur auf diese Hardware bezogen und können nicht in Bezug auf andere Systeme übertragen werden.

Beim OOD-Test ist zu benennen, dass der Test nur mit der 1fps Frame-Strategie und dem stärkeren Basismodell stattfand, da an ihm nur die Ergebnisse des Interpolationswertes erweitert werden sollten.

Zu benennen ist ebenfalls die Warmup-Fehlkonfiguration, welche zwar die interne Vergleichbarkeit nicht belastet, da die Konfiguration intern konsistent ist, aber die Aussage sollte zur Überprüfung ein weiteres Mal an optimal trainierten Modellen ausgeführt werden.

6.2 Ausblick

Aus den Befunden dieser Arbeit leiten sich einige Möglichkeiten für zukünftige wissenschaftliche Untersuchungen ab. Es sollte das Warmup an die Datenmenge angepasst werden, um zu prüfen, ob die gleichen Ergebnisse auch bei sauberem Fine-Tuning auftreten. Außerdem sollte der OOD-Test ausgeweitet werden, dabei könnten weitere Datensätze, sowie die unterschiedlichen Frame-Strategien und Basismodelle getestet werden. Der Datensatz für Training und Test könnte zudem günstig erweitert werden, indem weitere der vorhandenen Captions und mehr als einen Frame pro Video zum Trainieren verwendet werden würde.

Ein weiterer Optimierungsfaktor ist das Training der Netzwerke. Eine größere Batchsize und eine präzisere Lernrate könnten die Resultate des Fine-Tunings deutlich verbessern. Dabei sollte die Lernrate zu Beginn nicht zu hoch gewählt werden, da trotz der in dieser Arbeit verwendeten sehr niedrigen Lernrate Overfitting Erscheinungen auftraten.

Bei größeren verfügbaren Ressourcen sollte der Umfang der Arbeit erweitert werden, indem weitere und größere Modelle mit unterschiedlichen Architekturen und größere Datensätze verwendet werden. Das dürfte robustere und universellere Ergebnisse liefern. Dabei gäbe es ebenfalls die Möglichkeit, die Captions mit Vision-Language-Modellen einheitlicher zu erstellen und zu erweitern.
Bei steigender Datenmenge sollte geprüft werden, ob ein approximativer Index sinnvoller wäre, als der in dieser Arbeit verwendete exakte Index.

Bei längeren Videos könnte die szenenbasierte Sampling-Strategie deutlich besser abschneiden, da sie die relevanten Szenen besser erkennt. Zudem sollte in diesem Schritt die Möglichkeit in Betracht gezogen werden, mehrere Frames pro Szene zu verwenden, um die Repräsentation der Szenen zu verbessern.

Zur genaueren Untersuchung des α-Wertes könnte außerdem eine getrennte Evaluation von Image- und Text-Encodern stattfinden.

Zusammenfassend lässt sich festhalten, dass die Arbeit einen Beitrag zur Optimierung von CLIP-basierten Text-zu-Video-Retrieval-Systemen liefert, indem sie systematisch die Auswirkungen von Frame-Sampling-Strategien und Interpolationsfaktoren untersucht. Die entwickelten Methoden und gewonnenen Erkenntnisse führen zu Empfehlungen, die von denen in der Literatur verbreiteten Standardwerten abweichen. Die erzielten Gewinne hängen stark davon ab, wie weit Trainings- und Anwendungsdomäne auseinanderliegen. Für die Praxis bedeutet dies, dass fest gewählte Parameterwerte nicht zwingend die besten Ergebnisse liefern, sondern sich der Aufwand einer systematischen Untersuchung lohnt, um die Leistung des Systems zu optimieren. Die Ergebnisse beantworten die Forschungsfragen für den untersuchten Rahmen und liefern eine solide Grundlage für weiterführende Untersuchungen, die auf größeren Datensätzen, unterschiedlichen Modellen und erweiterten Captions basieren könnten.

Anhang A: Messergebnisse

Tabelle A.1: Vollständige Evaluationsergebnisse des WiSE-FT-Alpha-Sweeps. Alle Werte auf $n = 2990$ Text-Queries. $\alpha = 0{,}0$ entspricht dem Zero-Shot-Modell, $\alpha = 1{,}0$ dem fine-getunten Modell. (Formatiert mit Claude Opus 5)

α	R@1	R@5	R@10	MedR ↓	MnR ↓
datacomp_l, partielles FT (ep7), Keyframes 0,5 s					
0,0	0,1572	0,3428	0,4311	17,0	163,06
0,1	0,1622	0,3522	0,4388	16,0	153,97
0,2	0,1672	0,3592	0,4472	15,0	145,27
0,3	0,1672	0,3649	0,4528	14,0	136,85
0,4	0,1719	0,3692	0,4625	13,0	129,30
0,5	0,1726	0,3719	0,4709	13,0	122,86
0,6	0,1722	0,3799	0,4746	**12,0**	117,37
0,7	**0,1756**	0,3849	0,4773	**12,0**	112,87
0,8	0,1709	0,3873	**0,4829**	**12,0**	109,80
0,9	0,1732	**0,3900**	0,4826	**12,0**	107,77
1,0	0,1706	0,3836	0,4809	**12,0**	**106,70**
datacomp_l, partielles FT (ep7), Keyframes 1 s					
0,0	0,1592	0,3455	0,4338	16,0	162,94
0,1	0,1642	0,3532	0,4428	15,0	153,87
0,2	0,1686	0,3592	0,4555	14,0	145,13
0,3	0,1736	0,3625	0,4652	13,0	136,85
0,4	0,1773	0,3679	0,4756	12,0	129,43
0,5	0,1816	0,3793	0,4826	12,0	122,95
0,6	0,1829	0,3853	0,4870	**11,0**	117,23
0,7	**0,1860**	0,3896	0,4910	**11,0**	112,44
0,8	0,1826	0,3896	0,4936	**11,0**	109,01
0,9	0,1816	0,3906	**0,4943**	**11,0**	106,88
1,0	0,1819	**0,3926**	0,4876	**11,0**	**106,14**
datacomp_l, partielles FT (ep7), Keyframes 2 fps					
0,0	0,1602	0,3475	0,4365	16,0	163,34

Fortsetzung auf der nächsten Seite

Tabelle A.1: Vollständige Evaluationsergebnisse (Fortsetzung).

α	R@1	R@5	R@10	MedR ↓	MnR ↓
0,1	0,1645	0,3552	0,4441	15,0	154,21
0,2	0,1666	0,3602	0,4575	14,0	145,26
0,3	0,1672	0,3706	0,4669	13,0	136,93
0,4	0,1726	0,3756	0,4712	13,0	129,41
0,5	0,1776	0,3816	0,4766	12,0	123,06
0,6	0,1803	0,3846	0,4846	12,0	117,51
0,7	0,1793	0,3920	**0,4900**	**11,0**	112,96
0,8	**0,1816**	0,3913	0,4870	**11,0**	109,64
0,9	0,1799	**0,3936**	0,4870	12,0	107,27
1,0	0,1813	0,3906	0,4843	**11,0**	**106,24**
datacomp_l, partielles FT (ep7), Keyframes TransNetV2					
0,0	0,1458	0,3084	0,4027	21,0	185,39
0,1	0,1478	0,3147	0,4074	19,0	175,98
0,2	0,1485	0,3244	0,4151	18,0	166,83
0,3	0,1498	0,3321	0,4221	17,0	158,35
0,4	0,1502	0,3381	0,4278	17,0	150,71
0,5	0,1518	0,3455	0,4324	**16,0**	144,18
0,6	0,1542	0,3508	0,4371	**16,0**	138,97
0,7	0,1565	**0,3515**	0,4391	**16,0**	134,85
0,8	**0,1609**	0,3505	**0,4438**	**16,0**	131,75
0,9	0,1579	**0,3515**	**0,4438**	**16,0**	130,14
1,0	0,1552	0,3492	0,4435	**16,0**	**129,88**
datacomp_l, vollständiges FT (ep7), Keyframes 0,5 s					
0,0	0,1572	0,3428	0,4311	17,0	163,06
0,1	0,1639	0,3502	0,4415	16,0	152,38
0,2	0,1676	0,3575	0,4522	15,0	142,45
0,3	0,1712	0,3662	0,4615	14,0	133,27
0,4	0,1759	0,3712	0,4686	13,0	125,27
0,5	0,1789	0,3769	0,4773	12,0	118,50
0,6	0,1786	0,3826	0,4833	12,0	112,97
0,7	0,1786	0,3863	0,4873	**11,0**	108,74
0,8	**0,1799**	0,3910	0,4923	**11,0**	105,43
0,9	0,1793	**0,3940**	**0,4953**	**11,0**	103,13
1,0	0,1786	0,3923	0,4930	**11,0**	**101,81**
datacomp_l, vollständiges FT (ep7), Keyframes 1 s					
0,0	0,1592	0,3455	0,4338	16,0	162,94

Fortsetzung auf der nächsten Seite

Tabelle A.1: Vollständige Evaluationsergebnisse (Fortsetzung).

α	R@1	R@5	R@10	MedR ↓	MnR ↓
0,1	0,1645	0,3535	0,4448	15,0	152,29
0,2	0,1692	0,3605	0,4605	14,0	142,41
0,3	0,1739	0,3656	0,4726	13,0	133,35
0,4	0,1783	0,3729	0,4816	12,0	125,53
0,5	0,1856	0,3819	0,4860	12,0	118,94
0,6	0,1833	0,3940	0,4883	**11,0**	113,31
0,7	0,1856	0,3977	0,4957	**11,0**	108,57
0,8	0,1893	0,4013	**0,4983**	**11,0**	104,91
0,9	0,1880	**0,4020**	0,4980	**11,0**	102,35
1,0	**0,1923**	0,4003	**0,4983**	**11,0**	**101,01**
datacomp_l, vollständiges FT (ep7), Keyframes 2 fps					
0,0	0,1602	0,3475	0,4365	16,0	163,34
0,1	0,1652	0,3575	0,4448	15,0	152,63
0,2	0,1682	0,3652	0,4542	14,0	142,61
0,3	0,1709	0,3732	0,4692	13,0	133,47
0,4	0,1742	0,3799	0,4766	12,0	125,50
0,5	0,1779	0,3893	0,4823	12,0	118,84
0,6	0,1799	0,3923	0,4870	**11,0**	113,20
0,7	0,1836	0,3990	0,4933	**11,0**	108,45
0,8	**0,1876**	**0,4003**	**0,4967**	**11,0**	104,89
0,9	**0,1876**	**0,4003**	0,4960	**11,0**	102,36
1,0	0,1853	0,4000	0,4960	**11,0**	**100,76**
datacomp_l, vollständiges FT (ep7), Keyframes TransNetV2					
0,0	0,1458	0,3084	0,4027	21,0	185,39
0,1	0,1478	0,3167	0,4077	20,0	173,97
0,2	0,1488	0,3237	0,4214	18,0	163,17
0,3	0,1472	0,3318	0,4288	16,0	153,40
0,4	0,1505	0,3431	0,4375	16,0	145,10
0,5	0,1542	0,3498	0,4435	15,0	138,07
0,6	0,1582	0,3538	0,4508	**14,0**	132,43
0,7	0,1622	0,3615	0,4485	**14,0**	127,98
0,8	**0,1629**	**0,3652**	0,4552	**14,0**	124,57
0,9	0,1619	0,3645	**0,4565**	**14,0**	122,37
1,0	0,1602	0,3649	0,4528	**14,0**	**121,18**
datacomp_xl, partielles FT (ep7), Keyframes 0,5 s					
0,0	0,2074	0,4094	0,5033	10,0	127,87

Fortsetzung auf der nächsten Seite

Tabelle A.1: Vollständige Evaluationsergebnisse (Fortsetzung).

α	R@1	R@5	R@10	MedR ↓	MnR ↓
0,1	0,2130	0,4157	0,5117	9,5	119,48
0,2	0,2167	0,4227	0,5217	9,0	111,55
0,3	0,2174	0,4301	0,5281	8,5	104,15
0,4	0,2171	0,4378	0,5358	**8,0**	97,58
0,5	0,2211	0,4458	0,5391	**8,0**	92,06
0,6	0,2241	0,4468	0,5408	**8,0**	87,59
0,7	0,2247	**0,4485**	0,5411	**8,0**	84,09
0,8	**0,2251**	0,4468	0,5448	**8,0**	81,74
0,9	0,2244	0,4468	0,5472	**8,0**	80,27
1,0	0,2214	0,4475	**0,5478**	**8,0**	**79,78**
datacomp_xl, partielles FT (ep7), Keyframes 1 s					
0,0	0,2161	0,4120	0,5037	10,0	127,21
0,1	0,2227	0,4181	0,5151	10,0	118,94
0,2	0,2247	0,4244	0,5244	9,0	111,03
0,3	0,2271	0,4321	0,5308	9,0	103,67
0,4	0,2278	0,4355	0,5331	8,0	97,20
0,5	0,2314	0,4401	0,5415	8,0	91,90
0,6	0,2331	0,4438	0,5462	7,5	87,71
0,7	0,2351	0,4468	**0,5488**	**7,0**	84,41
0,8	**0,2375**	**0,4528**	**0,5488**	8,0	82,06
0,9	0,2331	0,4525	0,5448	7,5	80,73
1,0	0,2278	0,4522	0,5445	8,0	**80,37**
datacomp_xl, partielles FT (ep7), Keyframes 2 fps					
0,0	0,2137	0,4084	0,5037	10,0	128,43
0,1	0,2171	0,4167	0,5147	10,0	120,01
0,2	0,2191	0,4241	0,5204	9,0	112,01
0,3	0,2224	0,4344	0,5281	9,0	104,42
0,4	0,2244	0,4385	0,5338	8,0	97,96
0,5	0,2294	0,4448	0,5395	8,0	92,50
0,6	0,2318	0,4498	0,5445	8,0	88,21
0,7	0,2348	**0,4522**	0,5495	8,0	84,88
0,8	**0,2365**	0,4518	0,5522	**7,5**	82,46
0,9	0,2344	0,4492	0,5505	8,0	80,90
1,0	0,2294	0,4468	**0,5528**	8,0	**80,45**
datacomp_xl, partielles FT (ep7), Keyframes TransNetV2					
0,0	0,1766	0,3759	0,4702	12,0	144,67

Fortsetzung auf der nächsten Seite

Tabelle A.1: Vollständige Evaluationsergebnisse (Fortsetzung).

α	R@1	R@5	R@10	MedR ↓	MnR ↓
0,1	0,1789	0,3826	0,4826	12,0	136,16
0,2	0,1836	0,3873	0,4906	11,0	128,00
0,3	0,1860	0,3940	0,4963	11,0	120,29
0,4	0,1896	0,4017	0,4993	11,0	113,40
0,5	0,1983	0,4064	0,5054	**10,0**	107,56
0,6	0,2023	0,4077	0,5067	**10,0**	102,95
0,7	0,2054	0,4087	0,5090	**10,0**	99,39
0,8	0,2084	**0,4100**	**0,5151**	**10,0**	97,02
0,9	**0,2090**	**0,4100**	0,5114	**10,0**	95,74
1,0	0,2033	0,4064	0,5057	**10,0**	**95,46**
datacomp_xl, vollständiges FT (ep6), Keyframes 0,5 s					
0,0	0,2074	0,4094	0,5033	10,0	127,87
0,1	0,2124	0,4191	0,5130	10,0	118,99
0,2	0,2174	0,4258	0,5221	9,0	110,75
0,3	0,2201	0,4341	0,5264	9,0	103,18
0,4	0,2207	0,4421	0,5321	8,0	96,63
0,5	0,2227	0,4448	0,5361	8,0	91,24
0,6	0,2231	0,4488	0,5378	8,0	86,55
0,7	**0,2284**	0,4495	0,5421	8,0	82,79
0,8	0,2261	0,4512	0,5478	**7,0**	79,96
0,9	0,2261	0,4532	0,5482	**7,0**	78,11
1,0	0,2278	**0,4552**	**0,5488**	**7,0**	**76,93**
datacomp_xl, vollständiges FT (ep6), Keyframes 1 s					
0,0	0,2161	0,4120	0,5037	10,0	127,21
0,1	0,2207	0,4157	0,5171	10,0	118,46
0,2	0,2254	0,4217	0,5278	9,0	110,25
0,3	0,2294	0,4328	0,5328	8,0	102,87
0,4	0,2311	0,4391	0,5358	8,0	96,55
0,5	0,2321	0,4452	0,5425	8,0	91,11
0,6	0,2324	0,4515	0,5462	8,0	86,63
0,7	0,2365	0,4542	0,5482	**7,0**	83,17
0,8	0,2395	0,4582	0,5515	**7,0**	80,49
0,9	**0,2401**	0,4609	0,5528	**7,0**	78,68
1,0	0,2365	**0,4629**	**0,5548**	**7,0**	**77,63**
datacomp_xl, vollständiges FT (ep6), Keyframes 2 fps					
0,0	0,2137	0,4084	0,5037	10,0	128,43

Fortsetzung auf der nächsten Seite

Tabelle A.1: Vollständige Evaluationsergebnisse (Fortsetzung).

α	R@1	R@5	R@10	MedR ↓	MnR ↓
0,1	0,2174	0,4147	0,5144	10,0	119,52
0,2	0,2194	0,4241	0,5204	9,0	111,38
0,3	0,2221	0,4338	0,5254	9,0	103,77
0,4	0,2264	0,4378	0,5338	8,0	97,26
0,5	0,2304	0,4428	0,5408	8,0	91,89
0,6	0,2331	0,4515	0,5438	8,0	87,55
0,7	0,2331	0,4572	0,5475	**7,0**	84,10
0,8	0,2321	0,4592	0,5535	**7,0**	81,37
0,9	0,2338	0,4619	0,5535	**7,0**	79,50
1,0	**0,2351**	**0,4649**	**0,5542**	**7,0**	**78,38**
datacomp_xl, vollständiges FT (ep6), Keyframes TransNetV2					
0,0	0,1766	0,3759	0,4702	12,0	144,67
0,1	0,1799	0,3843	0,4806	12,0	135,65
0,2	0,1843	0,3903	0,4913	11,0	127,12
0,3	0,1883	0,4003	0,4943	11,0	119,27
0,4	0,1957	0,4090	0,4963	11,0	112,39
0,5	0,1973	0,4110	0,5017	10,0	106,58
0,6	0,2017	0,4107	0,5047	10,0	102,04
0,7	**0,2067**	0,4144	0,5104	10,0	98,54
0,8	0,2057	0,4157	0,5151	10,0	96,18
0,9	0,2023	**0,4204**	0,5171	**9,0**	94,77
1,0	0,2030	0,4197	**0,5207**	**9,0**	**94,15**

Abbildungsverzeichnis

2.1 Aufbau des Vision Transformer 9
2.2 Übersicht des gemeinsamen Vektorraums 9

3.1 Beispielvideos aus MSR-VTT 19
3.2 Beispielvideos aus Charades 20
3.3 Architektur des Python-Prototypen. Die gestrichelt umrandeten Komponenten bilden die in dieser Arbeit untersuchten Optimierungsachsen. 25

4.1 Screenshot von der Benutzeroberfläche der Streamlit-App vom 26.07.2026 . 36

5.1 Recall@1 über alle α; Formatierung mit Claude Opus 5 43
5.2 Recall@5 und Recall@10 über alle α; Formatierung mit Claude Opus 5 . . 43

Tabellenverzeichnis

3.1 Verwendete Technologien des Prototypen; (Formatiert mithilfe von Claude Opus 4.8) 24

4.1 Module des Prototypen mit ihren Aufgaben 27
4.2 Speicherbedarf der Frame-Sampling-Strategien für den gesamten MSR-VTT-Datensatz 29
4.3 Fine-Tuning Ergebnisse der zwei Strategien mit dem datacomp_l Checkpoint. (Tabellenlayout erstellt mit Gemini 3.6 Flash) 31
4.4 Fine-Tuning Ergebnisse der zwei Strategien mit dem datacomp_xl Checkpoint; (Tabellenlayout erstellt mit Gemini 3.6 Flash) 32
4.5 Speicheraufwand der Indizes beim MSR-VTT-Test-Split mit 2990 Videos . 34

5.1 Wirkung des Fine-Tunings bei 1fps auf dem MSR-VTT. 39
5.2 Lage des optimalen Interpolationsfaktors α je Konfiguration und Metrik. (Formatiert mit Gemini 3,6 Flash) 40
5.3 WiSE-FT Alpha-Sweep für `datacomp_l` (1fps). (Formatiert mit Claude Opus 5) 41
5.4 WiSE-FT Alpha-Sweep für `datacomp_xl` (1fps). (Formatiert mit Claude Opus 5) 41
5.5 R@1 in Abhängigkeit von α für über alle Frame-Strategien (Full-FT-Variante beider Modelle).(Formatiert mit Claude Opus 5) 42
5.6 Mittlere Differenz zum reinen Fine-Tuning ($\alpha = 1$), gemittelt über alle acht Konfigurationen je Modell.(Formatiert und berechnet mit Claude Opus 5) . 42
5.7 Beste erreichte Werte je Frame-Strategie 44
5.8 Anzahl der Frames im Index und mittlere Indexierungsdauer mit Performancevergleich zu R@1 45
5.9 Vergleich der `datacomp_l`- und `datacomp_xl`-Modellen bei 1fps. 46
5.10 Alpha-Sweep auf Charades mit `datacomp_xl` und 1fps.(Formatiert mit Claude Opus 5) 47
5.11 Zusammenfassung der Befunde mit Effektstärke in Prozentpunkten. Effektangaben beziehen sich auf R@1 in Prozentpunkten für jeweils die beste Konfiguration. (Formatiert mit Claude Opus 5) 49

A.1 Vollständige Evaluationsergebnisse des WiSE-FT-Alpha-Sweeps. Alle Werte auf $n = 2990$ Text-Queries. $\alpha = 0{,}0$ entspricht dem Zero-Shot-Modell, $\alpha = 1{,}0$ dem finegetunten Modell. (Formatiert mit Claude Opus 5) 55

Quelltextverzeichnis

4.1 Implementierung der WiSE-FT-Interpolation 33

Literaturverzeichnis

[1] Charu C. Aggarwal. *Neural Networks and Deep Learning: A Textbook*. Cham, Switzerland: Springer International Publishing, 2018. ISBN: 978-3-319-94462-3. DOI: 10.1007/978-3-319-94463-0. URL: https://link.springer.com/book/10.1007/978-3-319-94463-0.

[2] Tayfun Alpay u. a. „Multimodal video retrieval with CLIP: a user study“. In: *Information Retrieval Journal* 26.1 (Sep. 2023), S. 6. ISSN: 1573-7659. DOI: 10.1007/s10791-023-09425-2. URL: https://doi.org/10.1007/s10791-023-09425-2.

[3] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. New York, NY: Springer, 2006. ISBN: 978-0-387-31073-2.

[4] Christopher M. Bishop und Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. ISBN: 978-3-031-45467-7. DOI: 10.1007/978-3-031-45468-4. URL: https://link.springer.com/book/10.1007/978-3-031-45468-4.

[5] Mehdi Cherti u. a. „Reproducible scaling laws for contrastive language-image learning“. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2023, S. 2818–2829.

[6] Alexey Dosovitskiy u. a. „An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale“. In: *CoRR* abs/2010.11929 (2020). arXiv: 2010.11929. URL: https://arxiv.org/abs/2010.11929.

[7] Matthijs Douze u. a. *The Faiss library*. 2025. arXiv: 2401.08281 [cs.LG]. URL: https://arxiv.org/abs/2401.08281.

[8] Samir Yitzhak Gadre u. a. *DataComp: In search of the next generation of multimodal datasets*. 2023. arXiv: 2304.14108 [cs.CV]. URL: https://arxiv.org/abs/2304.14108.

[9] Matthias Hemmje und Stefan Wagenpfeil. *Multimedia Information Retrieval: Informationen aus multimedialen Quellen gewinnen*. 1. Aufl. Berlin, Heidelberg: Springer Vieweg, 2026, S. XXIII, 304. ISBN: 978-3-662-71886-5. DOI: 10.1007/978-3-662-71887-2.

[10] Ichiro Ide u. a., Hrsg. *MultiMedia Modeling: 31st International Conference, MMM 2025, Nara, Japan, January 8–10, 2025, Proceedings, Part I*. Springer Nature Singapore, 2025. ISBN: 978-981-96-2053-1. DOI: 10.1007/978-981-96-2054-8.

[11] Gabriel Ilharco u. a. *OpenCLIP*. Version 0.1. If you use this software, please cite it as below. Juli 2021. DOI: 10.5281/zenodo.5143773. URL: https://doi.org/10.5281/zenodo.5143773.

[12] Gabriel Ilharco u. a. *OpenCLIP*. Juli 2021. DOI: 10.5281/zenodo.5143773. URL: https://github.com/mlfoundations/open_clip (besucht am 02. 08. 2026).

[13] Huaishao Luo u. a. *CLIP4Clip: An Empirical Study of CLIP for End to End Video Clip Retrieval*. 2021. arXiv: 2104.08860 [cs.CV]. URL: https://arxiv.org/abs/2104.08860.

[14] Christopher D. Manning, Prabhakar Raghavan und Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[15] Nick Pantelidis u. a. „VERGE in VBS 2024“. In: *MultiMedia Modeling: 30th International Conference, MMM 2024, Amsterdam, The Netherlands, January 29 – February 2, 2024, Proceedings, Part IV*. Amsterdam, The Netherlands: Springer-Verlag, 2024, S. 356–363. ISBN: 978-3-031-53301-3. DOI: 10.1007/978-3-031-53302-0_32. URL: https://doi.org/10.1007/978-3-031-53302-0_32.

[16] Alec Radford u. a. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. arXiv: 2103.00020 [cs.CV]. URL: https://arxiv.org/abs/2103.00020.

[17] Olga Russakovsky u. a. *ImageNet Large Scale Visual Recognition Challenge*. 2015. arXiv: 1409.0575 [cs.CV]. URL: https://arxiv.org/abs/1409.0575.

[18] Konstantin Schall u. a. „Vibro: Video Browsing with Semantic and Visual Image Embeddings“. In: *MultiMedia Modeling: 29th International Conference, MMM 2023, Bergen, Norway, January 9–12, 2023, Proceedings, Part I*. Bergen, Norway: Springer-Verlag, 2023, S. 665–670. ISBN: 978-3-031-27076-5. DOI: 10.1007/978-3-031-27077-2_56. URL: https://doi.org/10.1007/978-3-031-27077-2_56.

[19] Klaus Schoeffmann, Daniela Stefanics und Andreas Leibetseder. „diveXplore at the Video Browser Showdown 2023“. In: März 2023, S. 684–689. ISBN: 978-3-031-27076-5. DOI: 10.1007/978-3-031-27077-2_59.

[20] Gunnar A. Sigurdsson u. a. *Hollywood in Homes: Crowdsourcing Data Collection for Activity Understanding*. 2016. arXiv: 1604.01753 [cs.CV]. URL: https://arxiv.org/abs/1604.01753.

[21] Alan F. Smeaton, Paul Over und Aiden R. Doherty. „Video shot boundary detection: Seven years of TRECVid activity“. In: *Computer Vision and Image Understanding* 114.4 (2010), S. 411–418. ISSN: 1077-3142. DOI: https://doi.org/10.1016/j.cviu.2009.03.011. URL: https://www.sciencedirect.com/science/article/pii/S1077314209000587.

[22] Tomáš Souček. *soCzech/TransNetV2*. original-date: 2019-10-20T16:20:02Z. Juli 2026. URL: https://github.com/soCzech/TransNetV2 (besucht am 02. 08. 2026).

[23] Tomáš Souček und Jakub Lokoč. „TransNet V2: An effective deep network architecture for fast shot transition detection“. In: *arXiv preprint arXiv:2008.04838* (2020).

[24] Kazuya Ueki u. a. „Optimization of CLIP Models for Domain-Specific Video Search“. In: *Proceedings of the 2025 International Conference on Multimedia Retrieval*. ICMR '25. Chicago, IL, USA: Association for Computing Machinery, 2025, S. 2053–2057. ISBN: 9798400718779. DOI: 10.1145/3731715.3733490. URL: https://doi.org/10.1145/3731715.3733490.

[25] *Video Browser Showdown – The Video Retrieval Competition*. en. URL: https://videobrowsershowdown.org/ (besucht am 16. 07. 2026).

[26] Zejia Weng u. a. *Open-VCLIP: Transforming CLIP to an Open-vocabulary Video Model via Interpolated Weight Optimization*. 2023. arXiv: 2302.00624 [cs.CV]. URL: https://arxiv.org/abs/2302.00624.

[27] Mitchell Wortsman u. a. *Robust fine-tuning of zero-shot models*. 2022. arXiv: 2109.01903 [cs.CV]. URL: https://arxiv.org/abs/2109.01903.

[28] Jun Xu u. a. „MSR-VTT: A Large Video Description Dataset for Bridging Video and Language“. In: IEEE International Conference on Computer Vision und Pattern Recognition (CVPR), Juni 2016. URL: https://www.microsoft.com/en-us/research/publication/msr-vtt-a-large-video-description-dataset-for-bridging-video-and-language/.

[29] Xiaohua Zhai u. a. *LiT: Zero-Shot Transfer with Locked-image text Tuning*. 2022. arXiv: 2111.07991 [cs.CV]. URL: https://arxiv.org/abs/2111.07991.